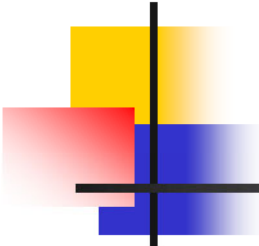


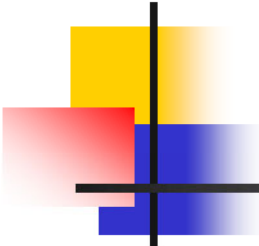


第七章 网络流



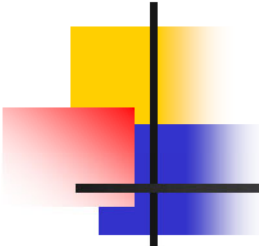
网络流

- 一个**二部图** $G=(V,E)$ 是一个无向图，结点集合可以划分成 $V=X\cup Y$ ，每条边 e 有一个端点在 X 中，一个端点在 Y 中。
- 图 $G=(V,E)$ 中的一个**匹配**是边的集合 $M\subseteq E$ ，并且每个结点至多出现在 M 中的一条边上。



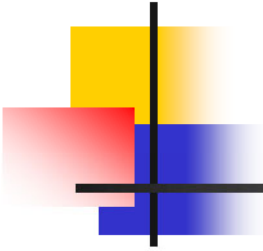
网络流

- 组合算法中最古老的问题：确定二部图中最大匹配的大小
- 可以有一个多项式时间的算法，但需要新的思路
- 本章中介绍一类一般化的问题-网络流问题，对一般性的问题，即最大流问题，开发一个多项式时间的算法，然后说明其一般性应用



网络流

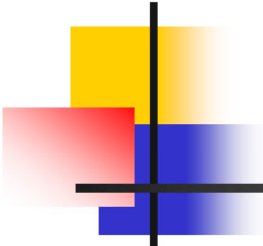
- 研究网络流问题的原始动力来自于网络的交通问题
- 当前解密的美军空军报告：揭示了在最小割应用的初始动机中，网络是一张前苏联的铁路线地图，目标是尽可能高效的破坏铁路运输



网络流

■ 应用场景

- ✓ Airline scheduling.
- ✓ Bipartite matching.
- ✓ Baseball elimination.
- ✓ Image segmentation.
- ✓ Network connectivity.
- ✓ Network reliability.
- ✓ ...



7.1 最大流问题

- 用图对交通网络建模
 - ✓ 比如公路系统：边是公路，结点交叉路口
 - ✓ 比如计算机网络：边是链接线，结点是开关
 - ✓ 比如管网：边是输送些体的管道，结点是管道连接点
- 抽象出来的要素：
 - 边上的容量；
 - 源点； 终点； 交通量通过边运送

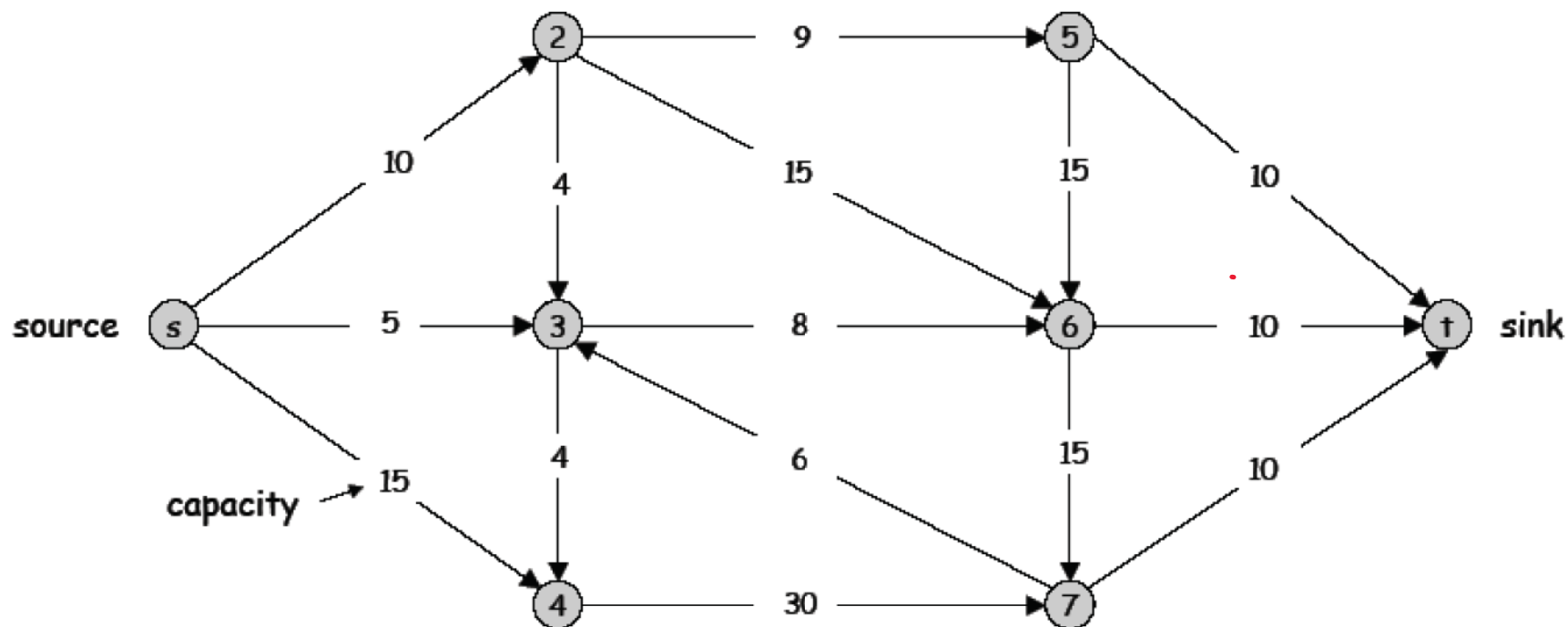
最大流问题

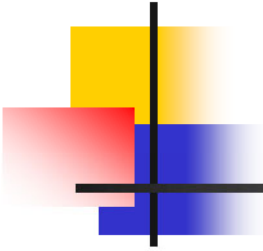
■ 流网络

有向图 $G=(V,E)$

每条边关联一个容量，非负数 C_e .

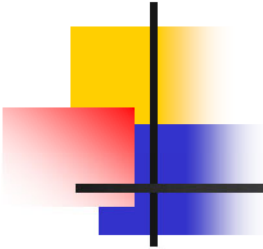
存在单一源点 s , 以及单一汇点 t





最大流问题

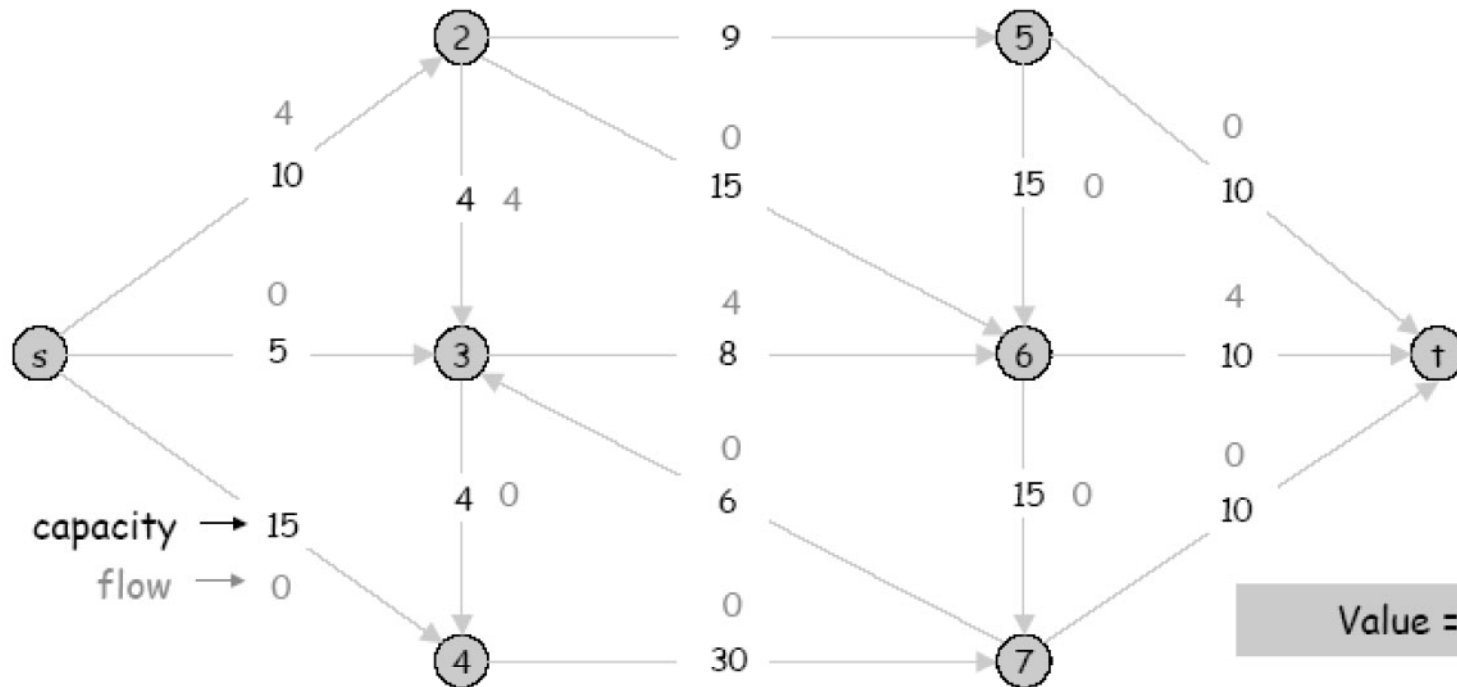
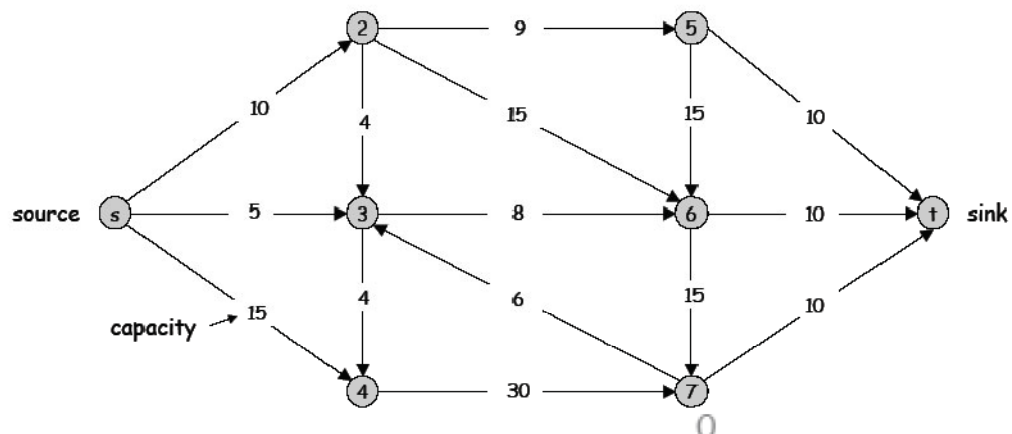
- **流**的定义:
- **s-t流**是一个函数 f ,它把每条边 e 映射到一个非负实数 $f: E \rightarrow R^+$,值 $f(e)$ 表示由边 e 携带的流量, 一个流 f 必须满足下面两个性质:
 1. (容量条件) $0 \leq f(e) \leq C_e$
 2. (守恒条件)除了 s, t 外, 对每个结点 v , 满足
$$\sum_{e \text{ in } v} f(e) = \sum_{e \text{ out } v} f(e)$$



最大流问题

- 定义 $f^{out}(v) = \sum_{e_{out_v}} f(e), f^{in}(v) = \sum_{e_{in_v}} f(e)$
- 我们记 $v(f) = f^{out}(s)$
- **最大流问题**：给定一个流网络，自然的目标就是安排交通使得有效容量尽可能得到有效使用：找出一个具有最大值的流

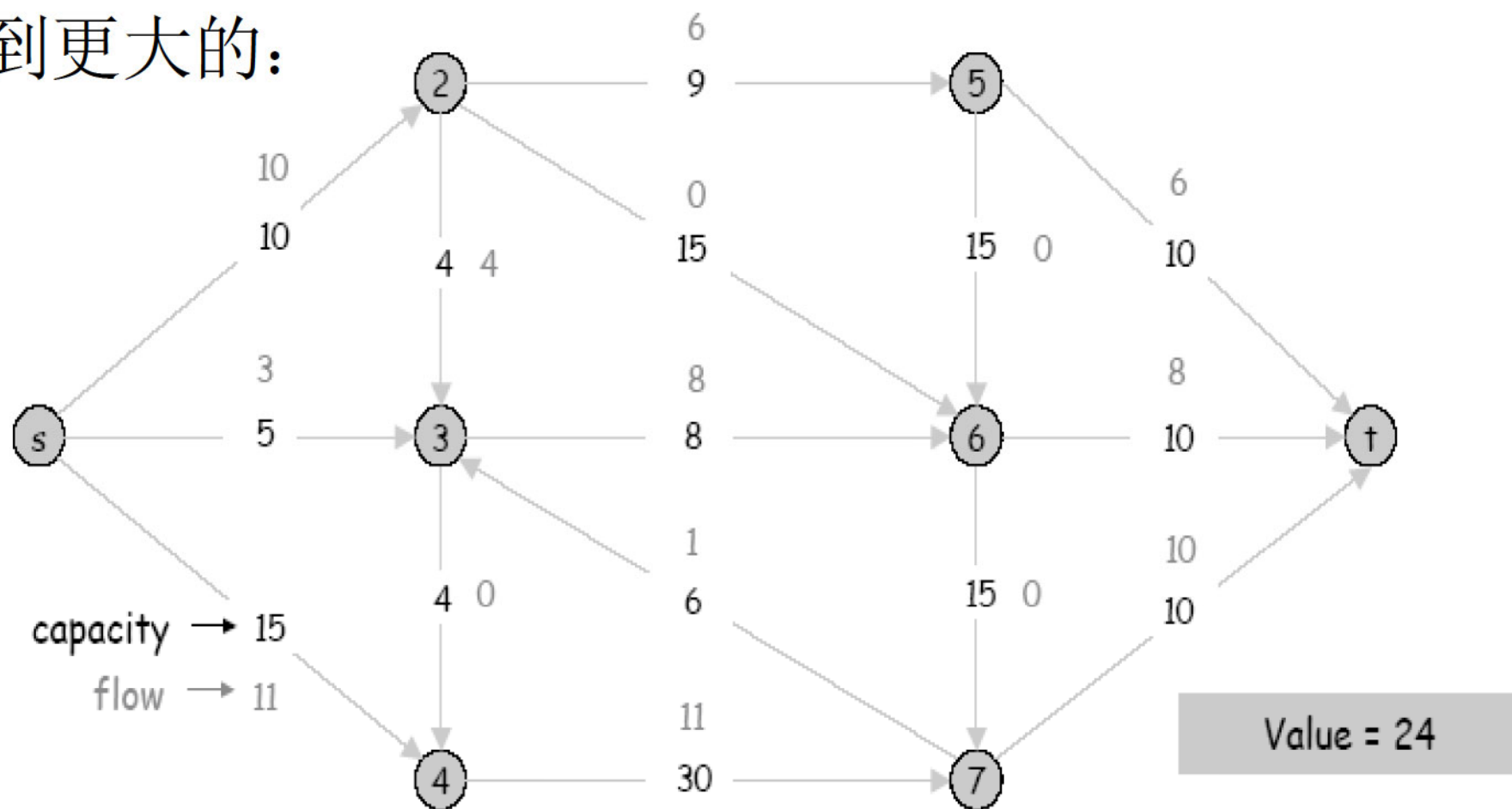
最大流问题



Value = 4

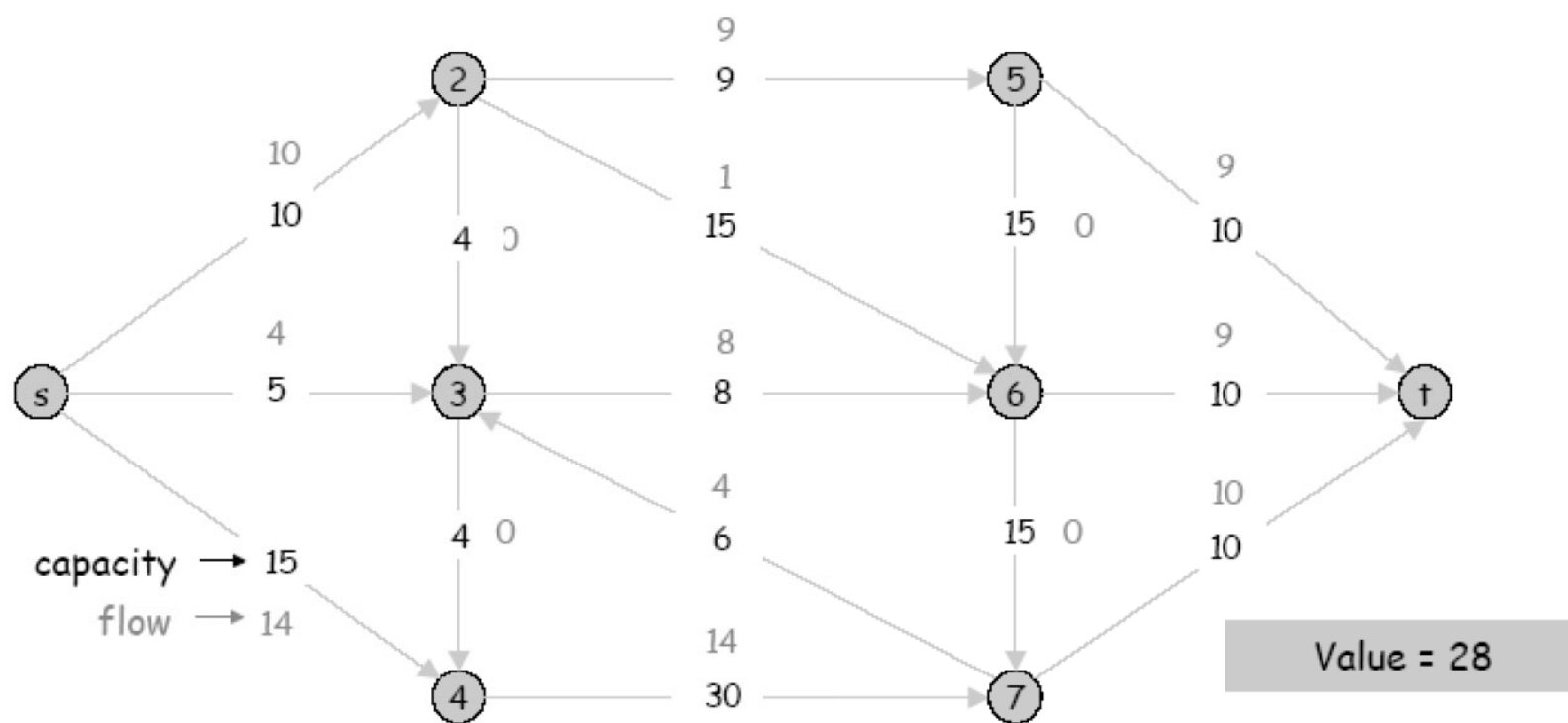
最大流问题

可以找到更大的:



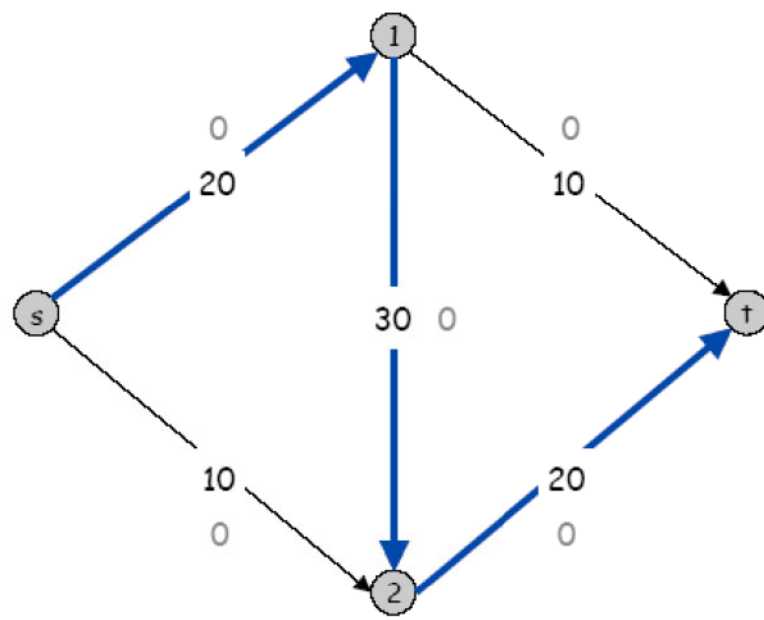
最大流问题

实际上最大的流:



最大流问题

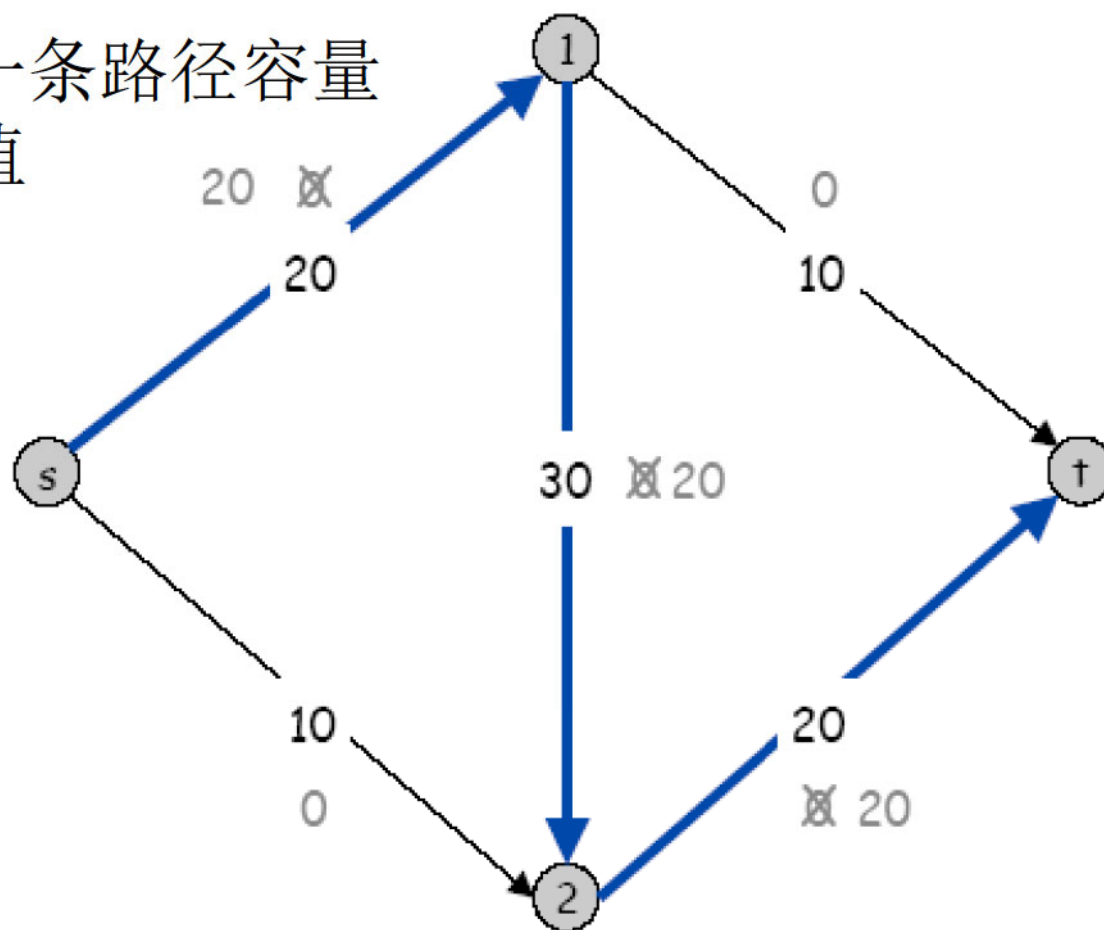
- 设计算法
- 先从贪心算法开始：对所有的 $e, f(e)=0$.
- 现在，沿着一条从 s 到 t 的路径通过“推”这个流来增加 f 的值，最大到边容量的限度。



Flow value = 0

最大流问题

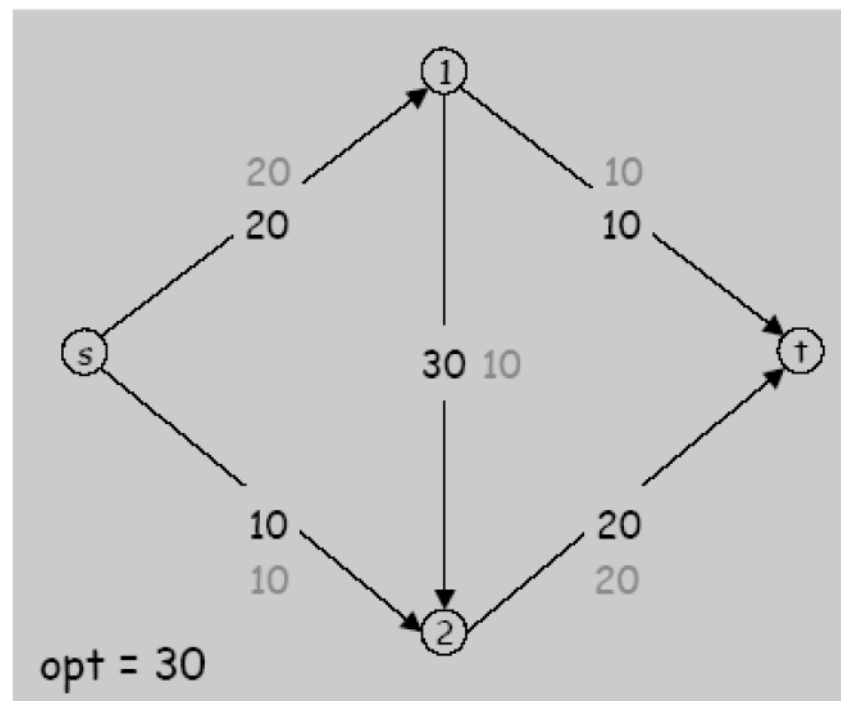
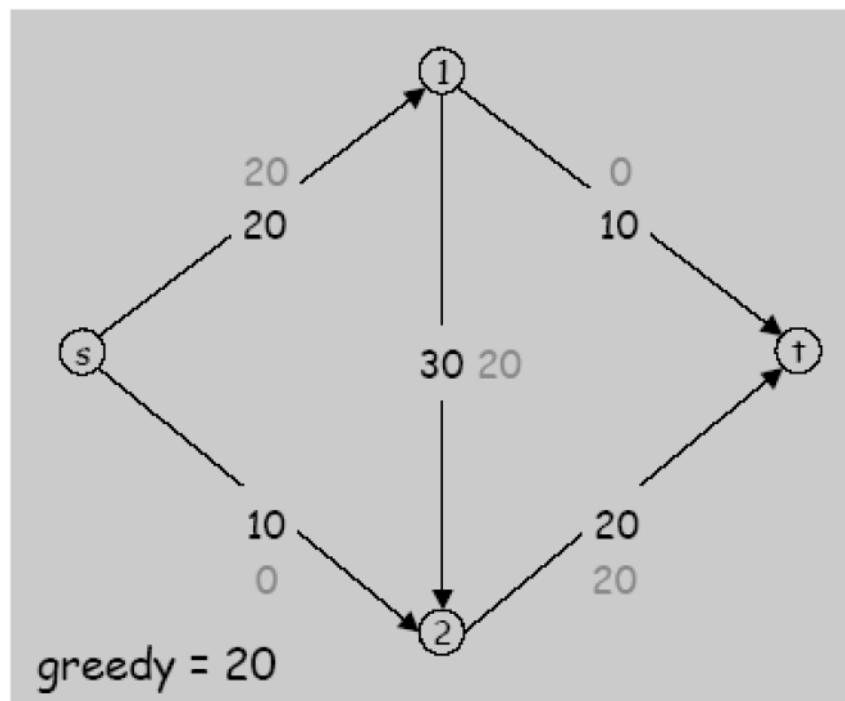
达到一条路径容量
最大值

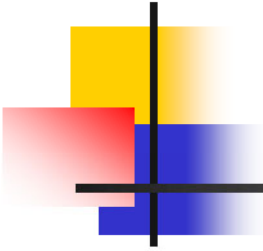


Flow value = 20

最大流问题

- 但是我们很快发现，局部最优不等于全局最优！





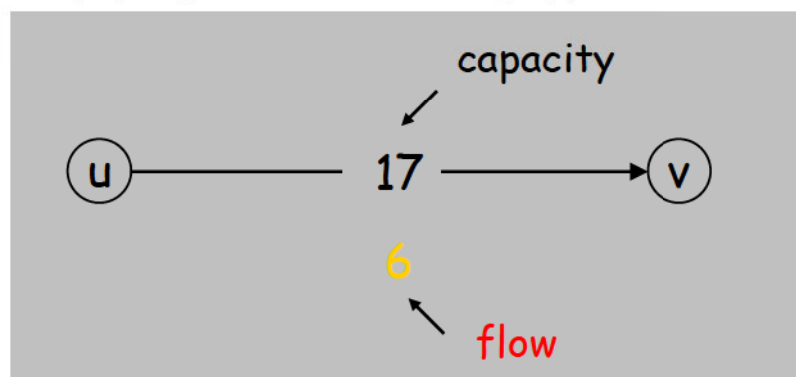
最大流问题

- 解决问题的思路
 - 分解
- 流的性质
- 我们可以在边上用剩余的容量向前推，并且我们可以在已经有流的边上向后推，使它转向一个不同的方向。
- 下面将引出剩余图的概念

最大流问题

- 原始边

$e = (u, v) \in E$, 流 $f(e)$, 容量 $c(e)$.



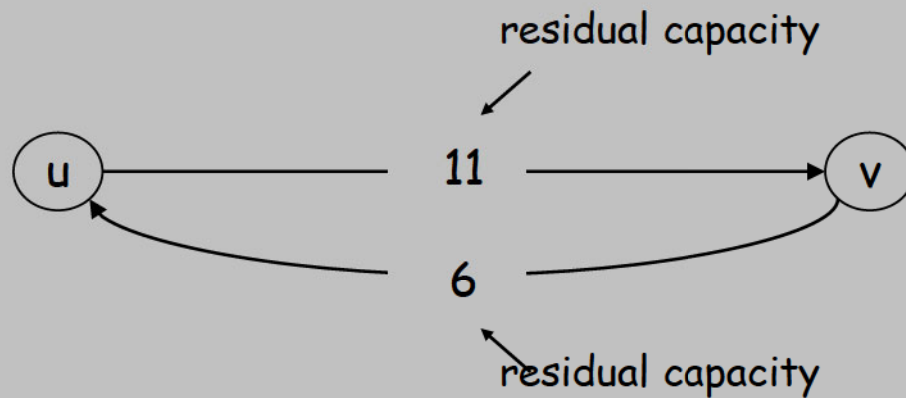
- 剩余边

$e = (u, v)$ and $e^R = (v, u)$.

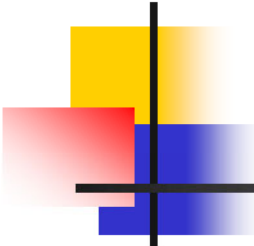
剩余容量:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{for } e \\ f(e) & \text{for } e^R \end{cases}$$

最大流问题



- 剩余图: $G_f = (V, E_f)$.
 - 具有正的剩余容量的剩余边.
 - $E_f = \{e\} \cup \{e^R\}$.



最大流问题

- 对 G 的每条边 $e=(u,v)$,其中 $f(e) < c(e)$,那么存在 $c(e)-f(e)$ 的剩余的容量,我们还可以尝试在这个容量往前推,于是 G_f 中包含这条边 e ,容量为 $c(e)-f(e)$,称为前向边。
- 对 G 的每条边 $e=(u,v)$,其中 $f(e) > 0$,我们可通过向后推这个流来“撤销”它,于是 G_f 中包含边 $e'=(v,u)$,容量是 $f(e)$,称为后向边。

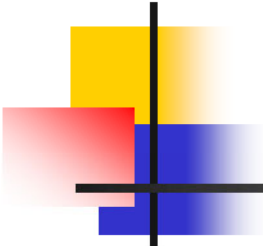
最大流问题

- 在剩余图中的增广路径
- 令 P 是 G_f 中一条简单的s-t路径。定义 $\text{bottleneck}(P, f)$ 是 P 上任何边关于流 f 的 最小剩余容量。如下算法 $\text{augment}(f, P)$ 在 G 中产生一个新的流 f' 。

```
Augment(f, P) {  
    b ← bottleneck(P)  
    foreach e ∈ P {  
        if (e ∈ E) f(e) ← f(e) + b  
        else      f(eR) ← f(e) - b  
    }  
    return f  
}
```

前向边

后向边



最大流问题

- 通常把剩余图中的任何一条**s-t**路径认为是一条**增广路径**。

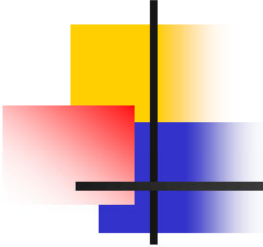
- 命题7.1 f' 是 G 中的一个流。

- 证明：验证容量条件与守恒条件。

对于前向边： $0 \leq f(e) \leq f'(e) = f(e) + \text{bottleneck}(P, f) \leq C_e$

对于后向边： $c_e \geq f(e) \geq f'(e) = f(e) - \text{bottleneck}(P, f) \geq 0$

守恒条件：分情形讨论



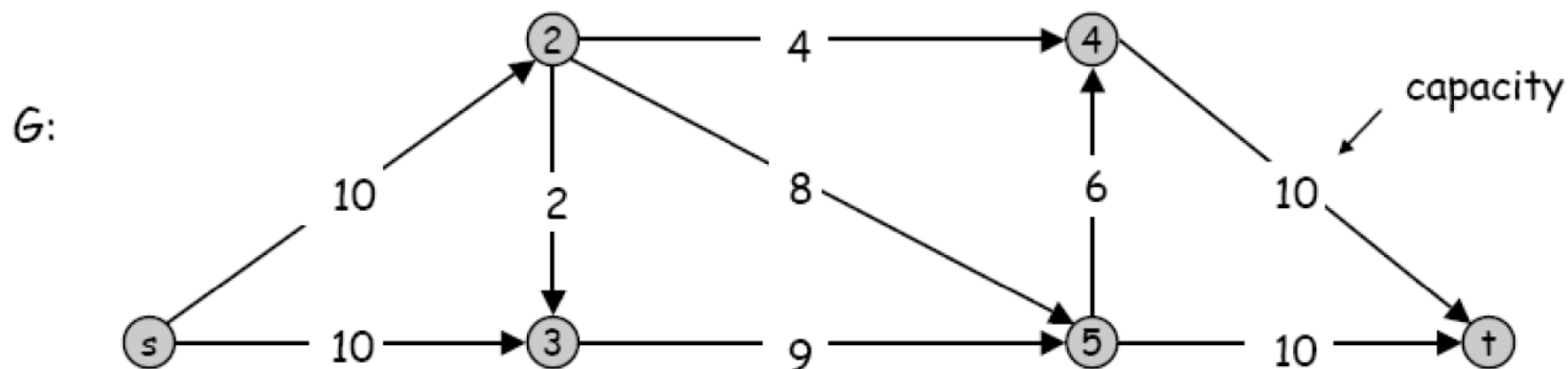
算法设计

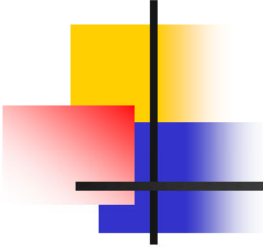
- 增广操作保持了向前和向后流的守恒性
- 直觉上告诉我们，可以不断调整 G_f 来获取更大的流量。

```
Ford-Fulkerson( $G, s, t, c$ ) {  
    foreach  $e \in E$   $f(e) \leftarrow 0$   
     $G_f \leftarrow$  residual graph  
  
    while (there exists augmenting path  $P$ )  
    {  
         $f \leftarrow$  Augment( $f, c, P$ )  
        update  $G_f$   
    }  
    return  $f$   
}
```

算法分析

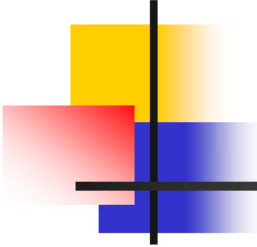
- Demo演示
- 初始图G





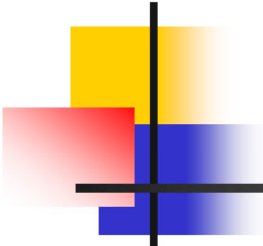
算法分析

- 1956, 由Ford, Fulkerson开发
- 正确性—确实最大(最大流与最小割)
- 算法复杂度--定量分析while循环在何时终止
- 命题7.2 在Ford-Fulkerson算法的每个中间步, 流值 $\{f(e)\}$ 和 G_f 中的剩余容量是整数。



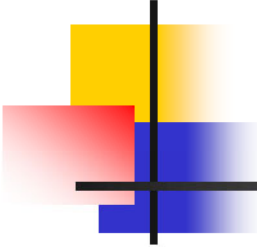
算法分析

- 命题7.3 令 f 是 G 中的流，且令 P 是 G_f 中的一条简单的 s - t 路径，那么 $v(f') = v(f) + \text{bottleneck}(P, f)$ ；并且由于 $\text{bottleneck}(P, f) > 0$ ，我们有 $v(f') > v(f)$ 。
- 证明： P 的第一条边 e 是从剩余图 G_f 中从 s 出来的边，边 e 一定是向前边。我们通过 $\text{bottleneck}(P, f)$ 增加了这条边上的流，且不改变其他的流。



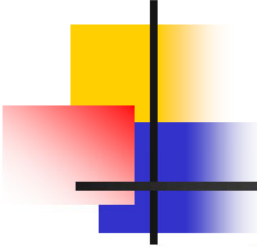
算法分析

- 最大可能的流值: $v(f) \leq C = \sum_{e \text{ out of } s} c_e$
- 因为有一个上界, 我们知道**Ford-Fulkerson**算法一定会终止
- **定理7.4** 如上所述, 假设在流网络**G**中的所有容量都是整数。那么**Ford-Fulkerson**算法在至多**C**次**While**循环的迭代后终止。



算法分析

- 下面考虑Ford-Fulkerson算法的运行时间。
- n 表示 G 中的结点数， m 表示 G 中的边数，所有的结点至少有一条关联边，于是 $m \geq n/2$;
- 算法复杂度？
- 定理7.5 假设在流网络 G 中的所有容量都是整数，那么Ford-Fulkerson算法可以在 $O(mC)$ 时间内实现



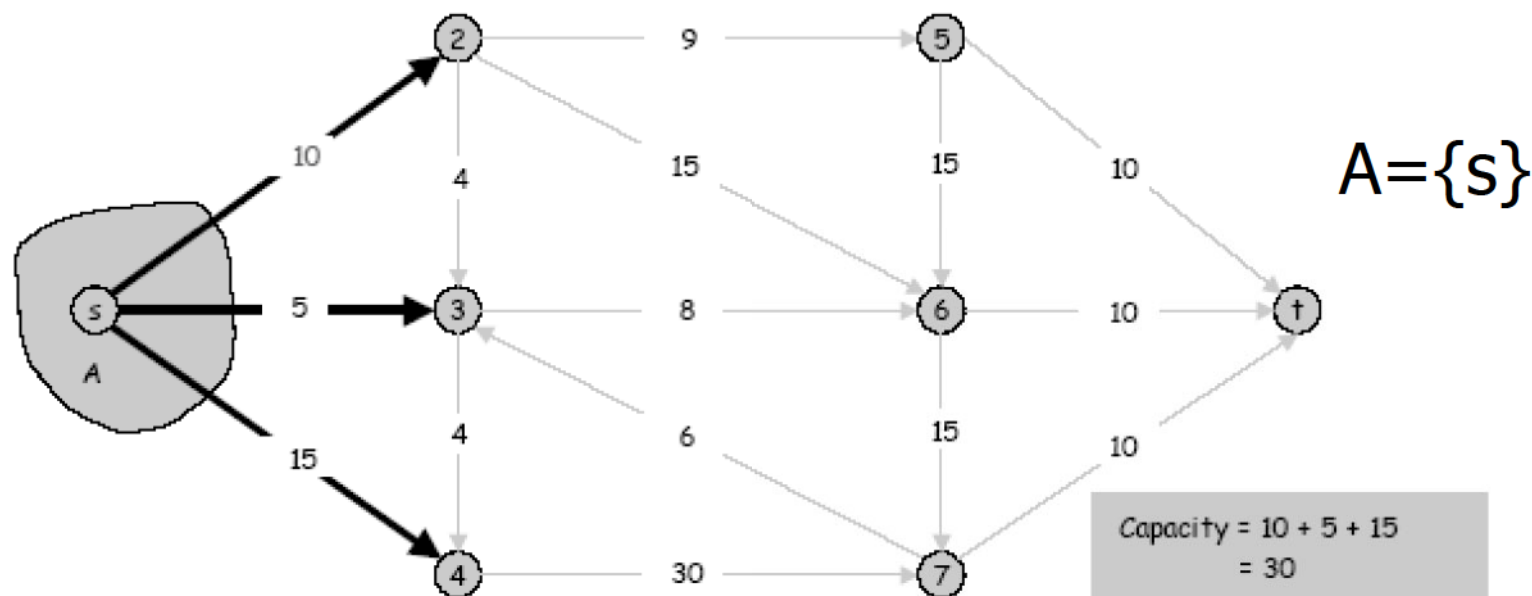
算法分析

■ 证明:

- 我们知道**While**循环至多在**C**次迭代后终止。于是考虑流调整一次时需要的复杂度;
- 剩余图至多有**2m**条边, 为找到 G_f 中一条s-t路径, 可以考虑宽度优先或者广度优先搜索, 代价为 $O(m+n)=O(m)$;
- 因为路径**P**至多有**n-1**条边, 建立新的剩余图需要 $O(m)$ 时间。

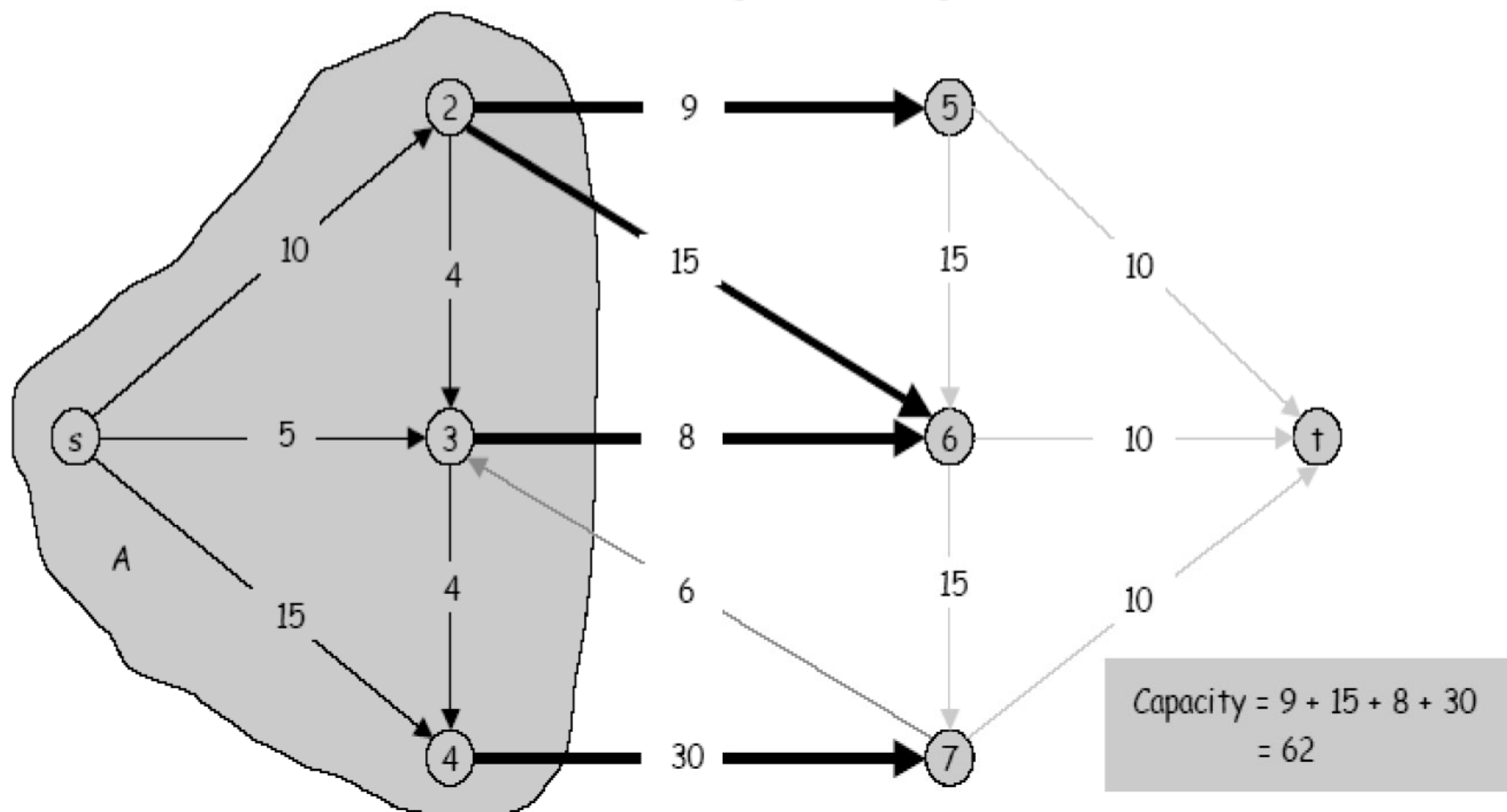
7.2 网络中的最大流与最小割

- 我们说一个**s-t**割是结点集合 V 的一个划分 (A, B) , 使得 $s \in A, t \in B$. 一个割 (A, B) 的容量记为 $c(A, B)$. 也就是从 A 出来的所有边的容量之和。



最大流与最小割

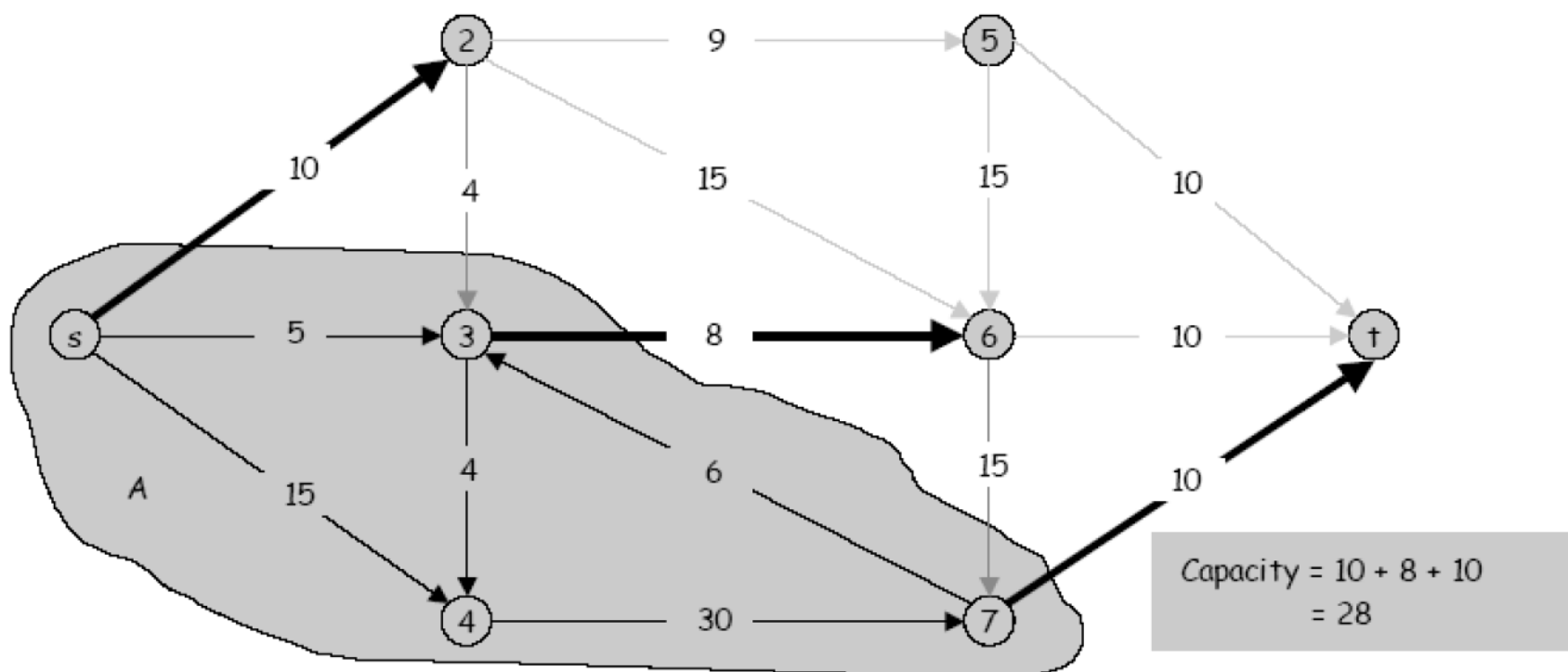
另外一种割的划分: $A = \{s, 2, 3, 4\}$

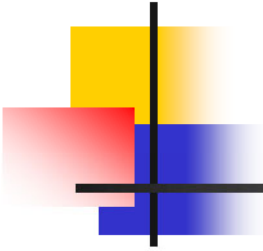


最大流与最小割

最小s-t割问题

- 寻找一个最小容量的 s-t 割。





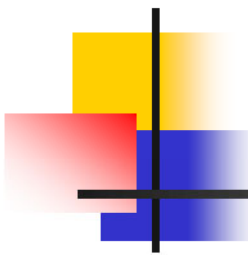
最大流与最小割

- 割原来提供了流值上的非常自然的上界
- 定理7.6 令 f 是任何 s - t 流，且 (A,B) 是任意 s - t 割，那么 $v(f) = f^{out}(A) - f^{in}(A)$.
- 证明：因为源点 s 没有边进入，所以

$$v(f) = f^{out}(s) - f^{in}(s)$$

此外其他 v 是内点，所以 $v(f) = \sum_{v \in A} (f^{out}(v) - f^{in}(v))$

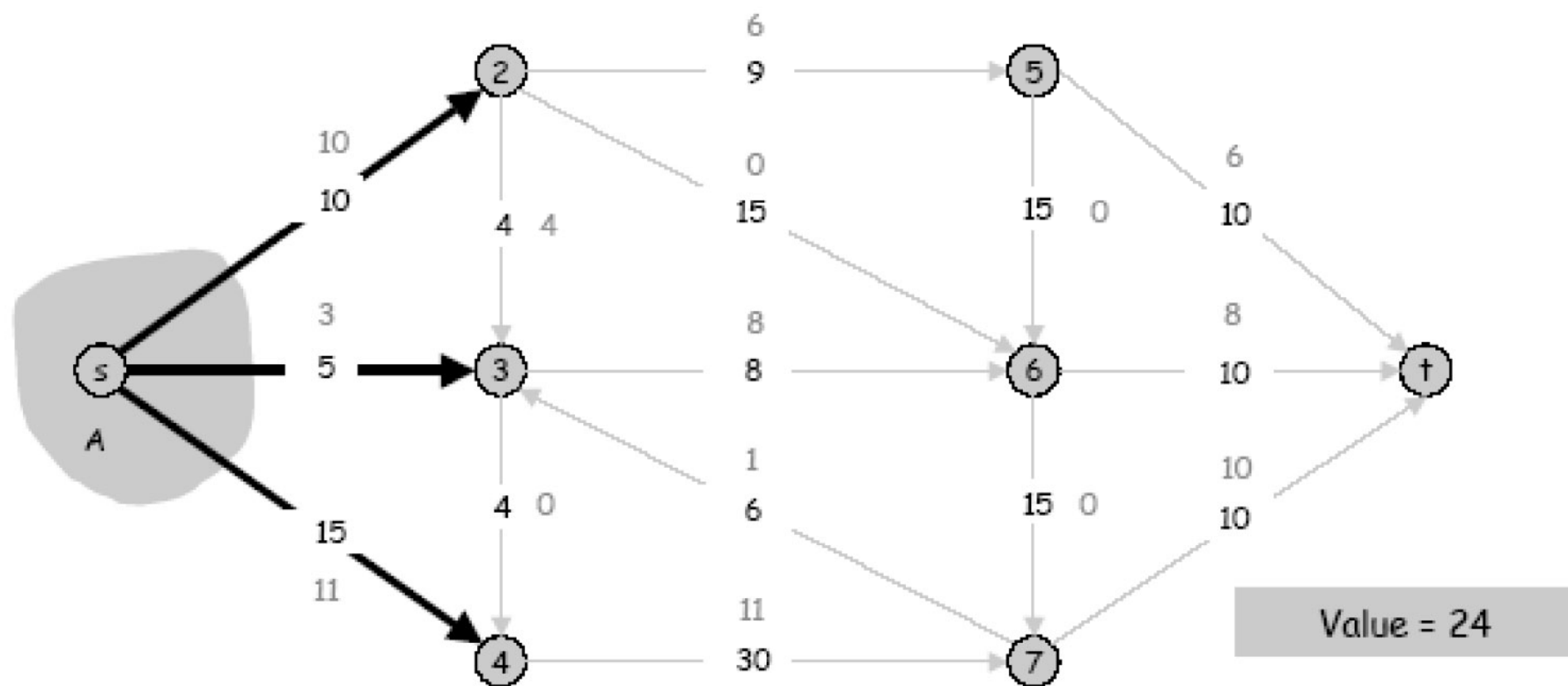
注意到 $\sum_{v \in A} (f^{out}(v) - f^{in}(v)) = \sum_{e_{out_A}} f(e) - \sum_{e_{in_A}} f(e) = f^{out}(A) - f^{in}(A)$



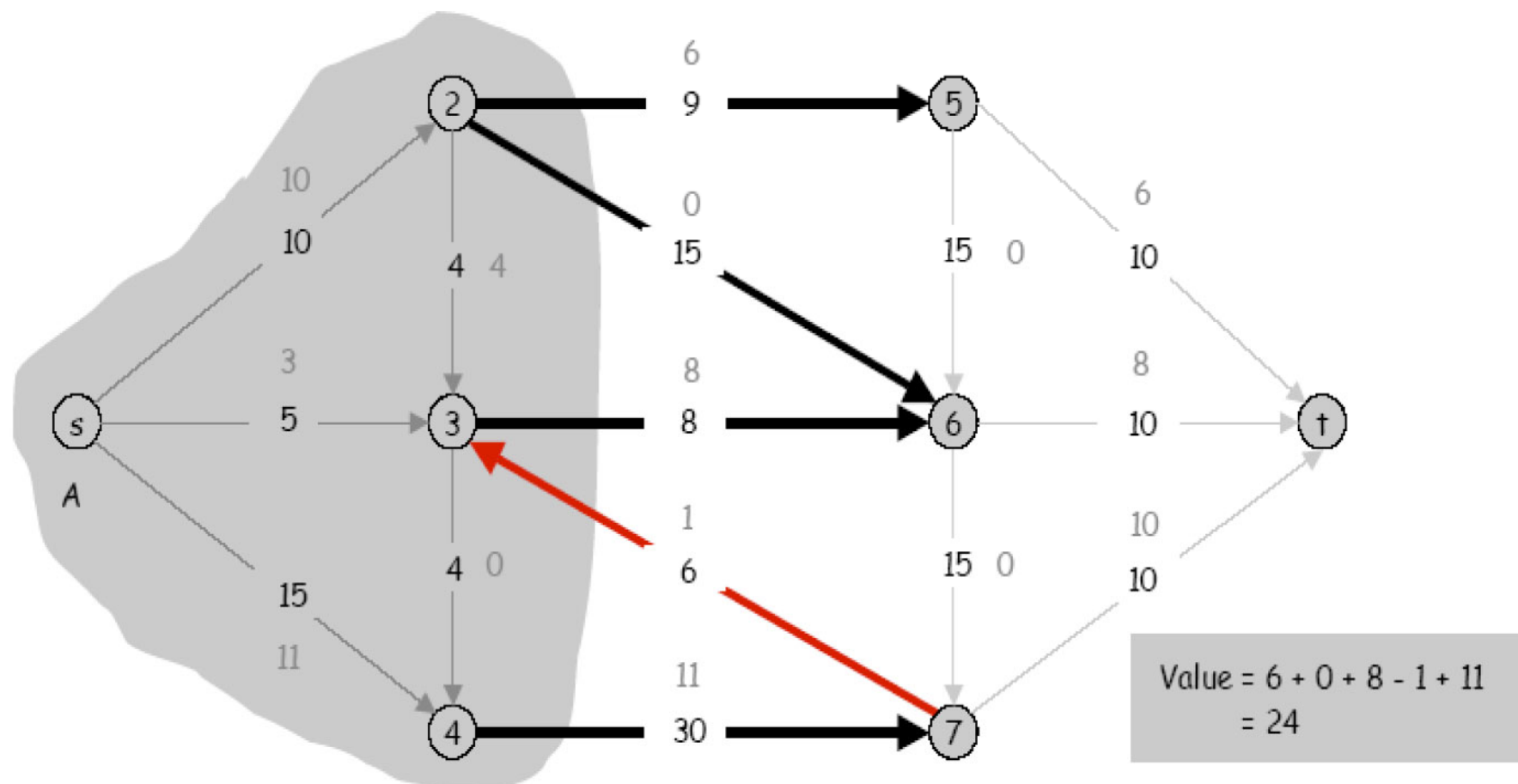
最大流与最小割

- 命题7.7 令 f 是任意 s - t 流，且 (A,B) 是任意 s - t 割，那么 $v(f) = f^{in}(B) - f^{out}(B)$

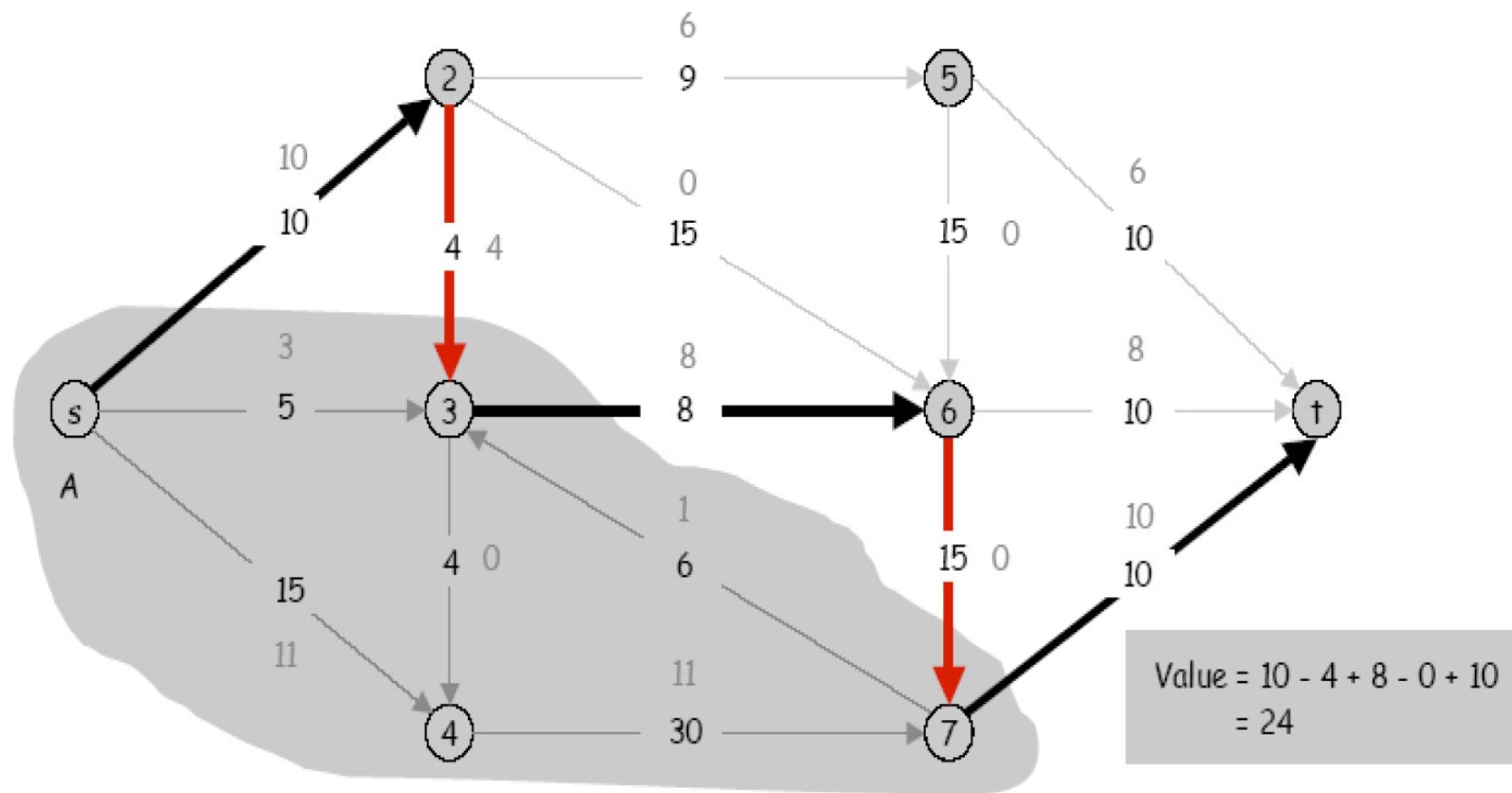
最大流与最小割



最大流与最小割



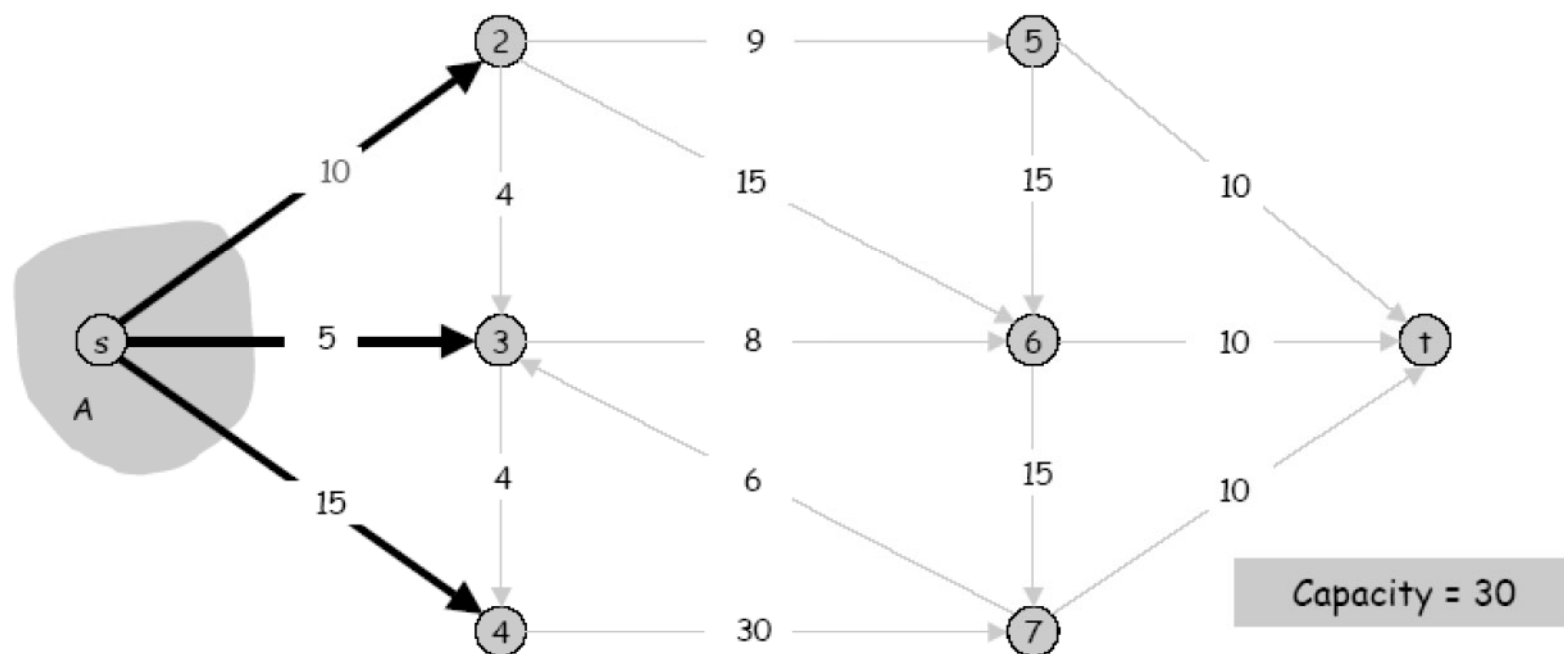
最大流与最小割



最大流与最小割

- 定理7.8 令 f 是任意 s - t 流, 且 (A,B) 是任意 s - t 割, 那么 $v(f) \leq c(A,B)$

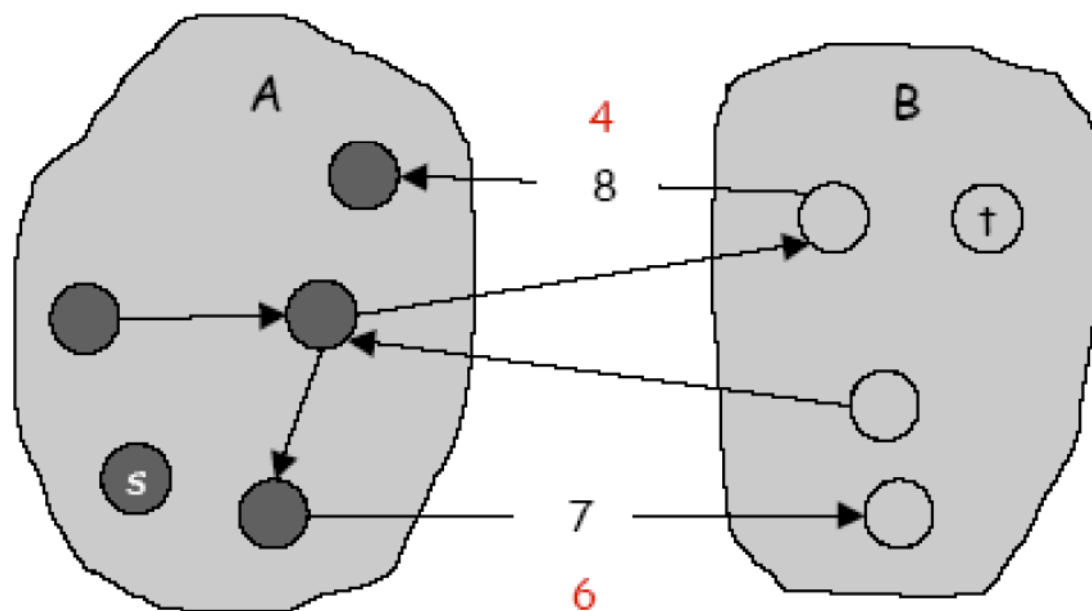
Cut capacity = 30 $\Rightarrow$ Flow value ≤ 30

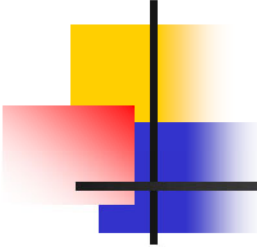


最大流与最小割

■ 证明:

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &\leq \sum_{e \text{ out of } A} f(e) \\ &\leq \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \quad \blacksquare \end{aligned}$$





最大流与最小割

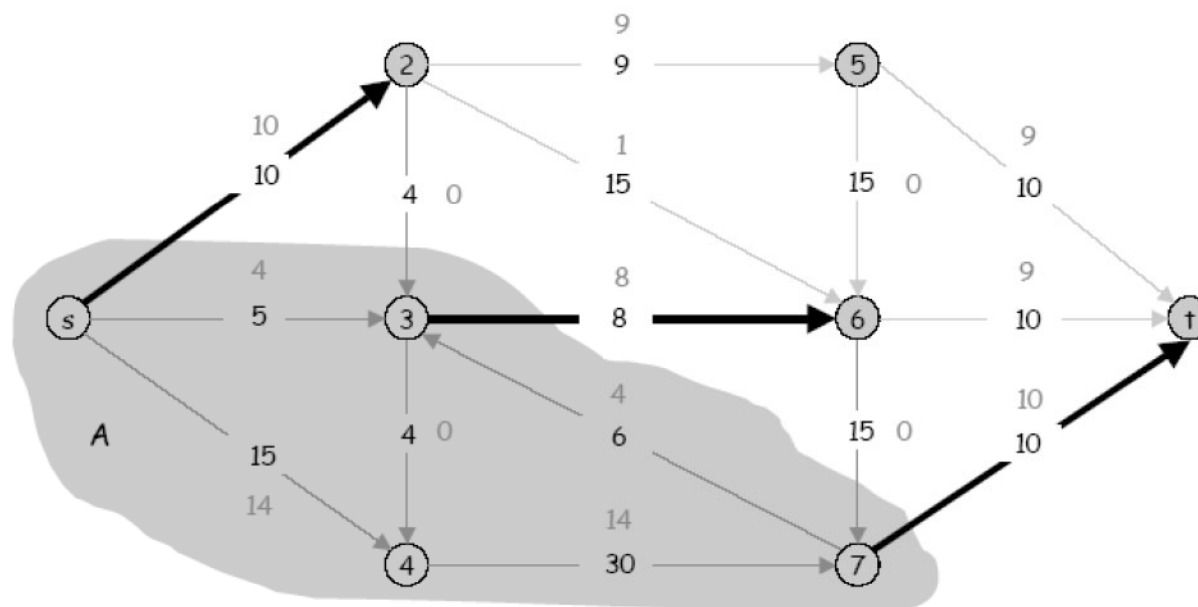
- 定理**7.8**说明，每个流的值是以每个割的容量为上界的
- 如果我们知道**G**中具有某个值 **c^*** 的**s-t**割，可以推出**G**中不可能有任何值大于 **c^*** 的**s-t**流
- 如果我们知道**G**中具有某个值 **v^*** 的**s-t**流，可以推出**G**中不可能有任何值小于 **v^*** 的**s-t**割

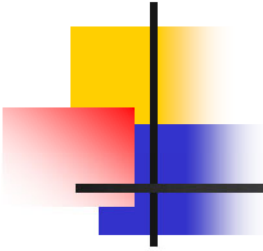
最大流与最小割

- 推论. 设 f 是任意的流, 并设 (A, B) 是任意的割. 如果 $v(f) = \text{cap}(A, B)$, 那么 f 是最大流, 并且 (A, B) 是最小割.

Value of flow = 28

Cut capacity = 28 $\Rightarrow$ Flow value ≤ 28

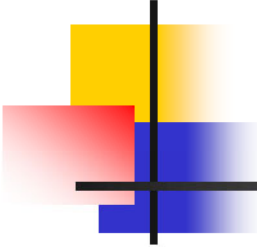




最大流与最小割

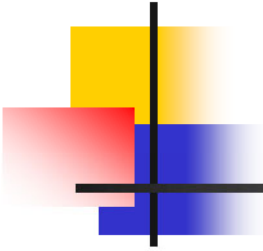
- 令 $\bar{f}$ 表示由Ford-Fulkerson返回的流
- 下面我们将给出一个s-t割 (A^*, B^*) 使得
$$v(\bar{f}) = c(A^*, B^*)$$
这直接说明 $\bar{f}$ 有任何流的最大值，并且 (A^*, B^*) 有任何s-t割最小的容量

Ford-Fulkerson终止时的流有什么性质？



最大流与最小割

- 定理7.9 如果 f 是使得剩余图 G_f 中没有 s - t 路径的一个 s - t 流, 那么在 G 中存在一个 s - t 割 (A^*, B^*) 使得 $v(f) = c(A^*, B^*)$. 因此, f 有 G 中任何流的最大值, 且 (A^*, B^*) 有 G 中任何 s - t 割的最小容量。
- 最大流最小割定理. [Ford-Fulkerson 1956] 最大流的值等于最小割



最大流与最小割

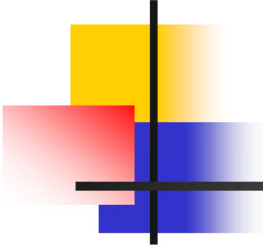
- 证明思路:

- (i) 存在 cut (A, B) 使得 $v(f) = \text{cap}(A, B)$.

- (ii) 流 f 是一个最大流.

- (iii) f 中没有增广路径.

(i) $\Rightarrow$ (ii) $\Rightarrow$ (iii) $\Rightarrow$ (i)



最大流与最小割

- (i) $\Rightarrow$ (ii) 显而易见
- (ii) $\Rightarrow$ (iii) 运用反证法. 设 f 是一个流, 如果还存在一条增广路经, 那么我们还可以继续改进 f , 矛盾。
- (iii) $\Rightarrow$ (i)
实际上这是算法停止运行的条件

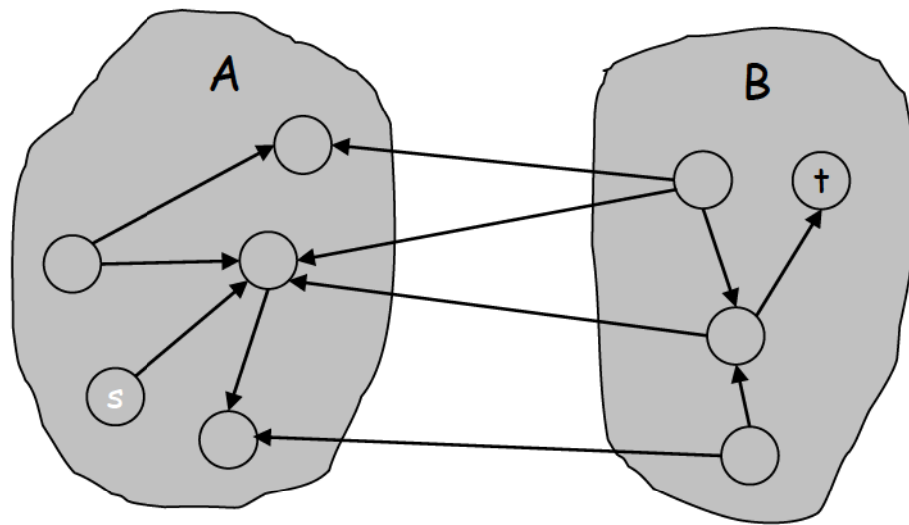
最大流与最小割

- 设流 f 没有增广路径.
- 定义集合 A 是剩余图 G_f 中从源点 s 可达顶点集合.
- 根据定义, 那么 $s \in A$; 终点 $t \notin A, \in B$.

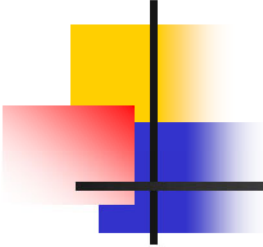
如果 $e=(u,v), u \in A, v \in B$, 那么 $f(e)=c(e)$;

如果 $e'=(u',v'), u' \in B, v' \in A$, 那么 $f(e')=0$.

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &= \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \end{aligned}$$

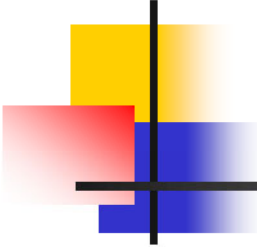


剩余图



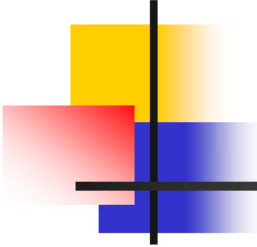
最大流与最小割

- 定理7.10 由Ford-Fulkerson算法返回的流 $\bar{f}$ 是最大流。
- 给定 $\bar{f}$ ，计算最小s-t割(A,B)的时间？



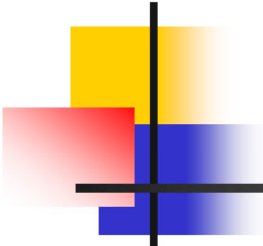
最大流与最小割

- 定理7.11 给定一个最大值的流 $\bar{f}$ ，我们可以在 $O(m)$ 时间内计算一个最小容量的 **s-t** 割。
- 命题7.12 在每个流网络中，存在一个流 **f** 和一个割 (A, B) ，使得 $v(f) = c(A, B)$ 。
- 定理7.14 如果在流网络中所有的容量都是整数，那么存在一个最大流 **f**，它的每个流值 **f(e)** 都是整数。



最大流与最小割

- 如果边的权值(容量)是实数，最大流最小割定理依然成立。
- 但是由于增广路径选择不合理，具有实数容量的**Ford-Fulkerson**算法可能永远运行下去
- 选择好的增广路径

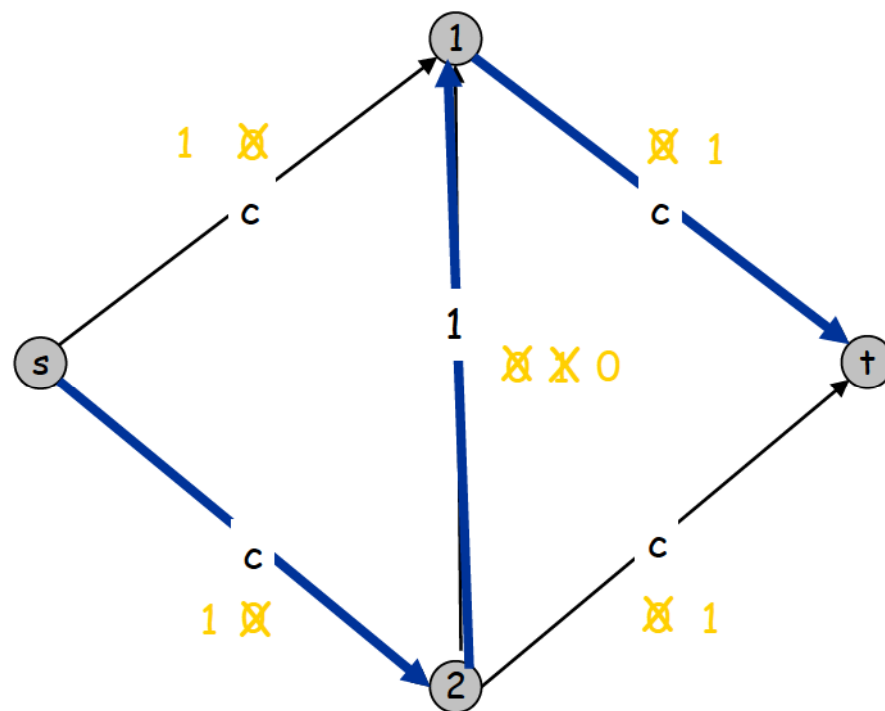
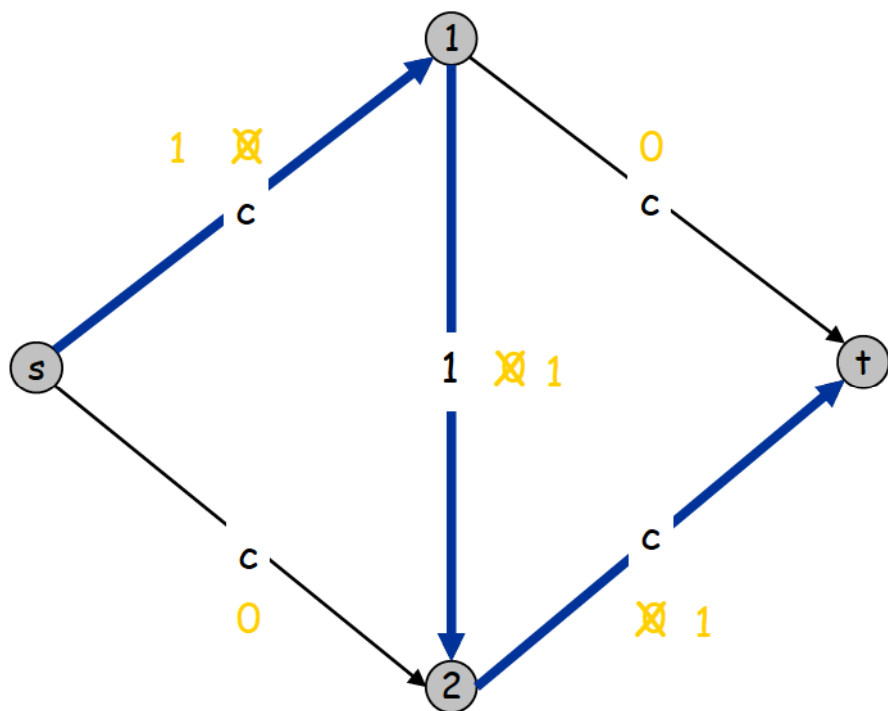


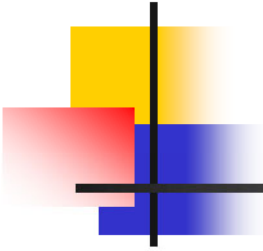
7.3 选择好的增广路径

- 一般的Ford-Fulkerson算法复杂度是不是输入规模的多项式时间？(输入数据： $m, n, \log C$)
- 不是，定理7.5 告诉我们，Ford-Fulkerson算法的运行时间在 $O(mC)$ ，伪多项式时间
- 有些时候算法的执行会非常的没有效率

选择好的增广路径

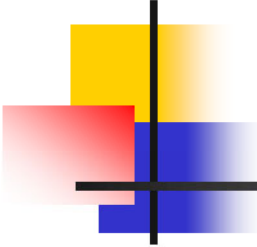
- 如果最大的流容量是 C , 算法可能要循环 C 次





选择好的增广路径

- 之所以出现这样的问题，在于我们刚才选择了一条**瓶颈容量**很小的增广路径，导致**收敛**很慢
- 所以我们的思路是：
因为增广路径通过选择路径的瓶颈容量来增加最大流的值，我们选择**具有大的瓶颈容量**的路径，那么算法进展会更大些

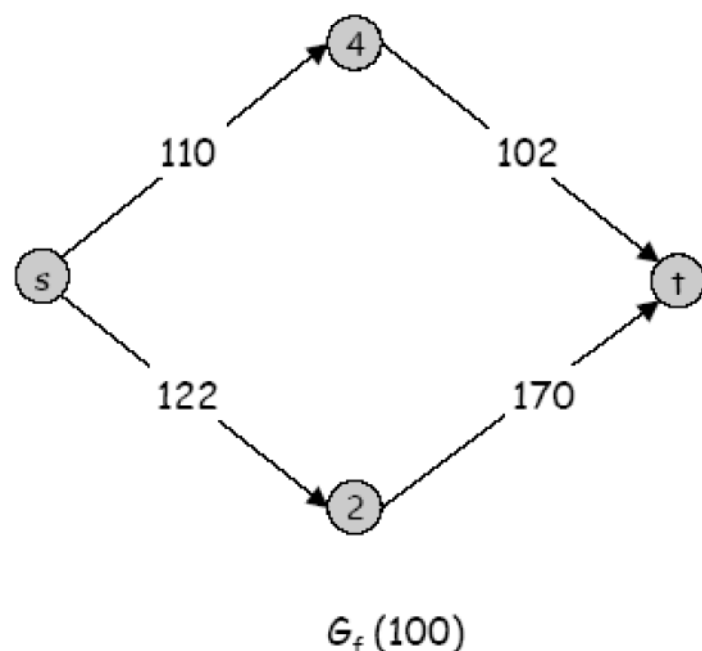
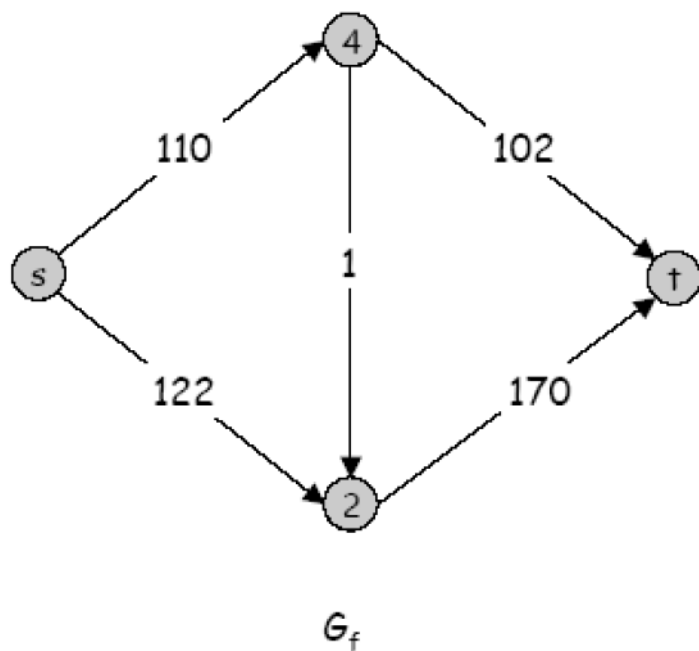


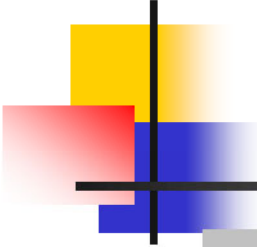
选择好的增广路径

- 选择好的增广路径：
Sufficiently large bottleneck capacity
- 这里为了便于控制，我们维护一个称之为缩放参数的 Δ ，算法中将寻找瓶颈容量至少是 Δ 的路径。

选择好的增广路径

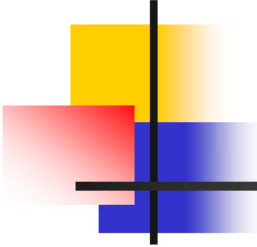
令 $G_f(\Delta)$ 是仅由剩余容量至少为 Δ 的边组成的
剩余图的子集.





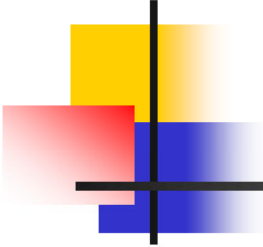
算法

```
Scaling-Max-Flow( $G, s, t, c$ ) {  
  foreach  $e \in E$   $f(e) \leftarrow 0$   
   $\Delta \leftarrow$  smallest power of 2 greater than or equal to  $C$   
   $G_f \leftarrow$  residual graph  
  
  while ( $\Delta \geq 1$ ) {  
     $G_f(\Delta) \leftarrow \Delta$ -residual graph  
    while (there exists augmenting path  $P$  in  $G_f(\Delta)$ ) {  
       $f \leftarrow$  augment( $f, c, P$ )  
      update  $G_f(\Delta)$   
    }  
     $\Delta \leftarrow \Delta / 2$   
  }  
  return  $f$   
}
```

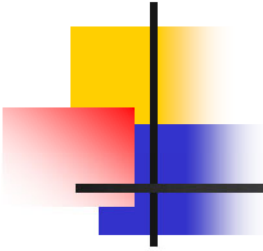
算法分析

- 流在算法中始终保持整数值，因此所有的剩余容量也是整数值。
- 定理7.15 如果容量是整数值，那么在缩放最大流算法中流和剩余容量也始终保持整数值，这就推出当 $\Delta = 1$ ， $G_f(\Delta) = G_f$ ，算法终止时，流 f 是最大值的流。



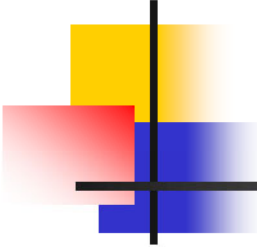
算法分析

- 现在我们开始关注算法的循环部分，估计各部分循环的次数
- 最外层循环While的次数？
- 命题7.16 外层While循环的迭代次数至多是 $1 + \lceil \log_2 C \rceil$
证明： 最开始 $C \leq \Delta < 2C$ ， Δ 每次缩小一半



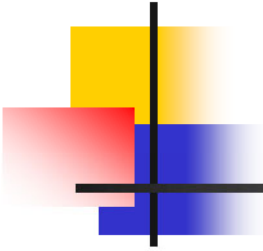
算法分析

- 下面需要界定内层循环在每个缩放阶段所做的增广次数
- 在 Δ 缩放阶段，我们只用到剩余容量至少是 Δ 的边，根据算法每次增加瓶颈容量的性质，就有
- 命题7.17 在 Δ 缩放阶段，每次增广增加的流值至少是 Δ .



算法分析

- 另外关键点一点是 Δ 缩放阶段结束时, $v(f)$ 不可能距最大值太远
- 命题7.18 令 f 是 Δ 缩放阶段结束时的流. G 中 存在一个 s - t 割 (A,B) 使得 $c(A,B) \leq v(f) + m \Delta$, 其中 m 是图 G 中的边数。因此在网络中的最大流值至多是 $v(f) + m \Delta$.

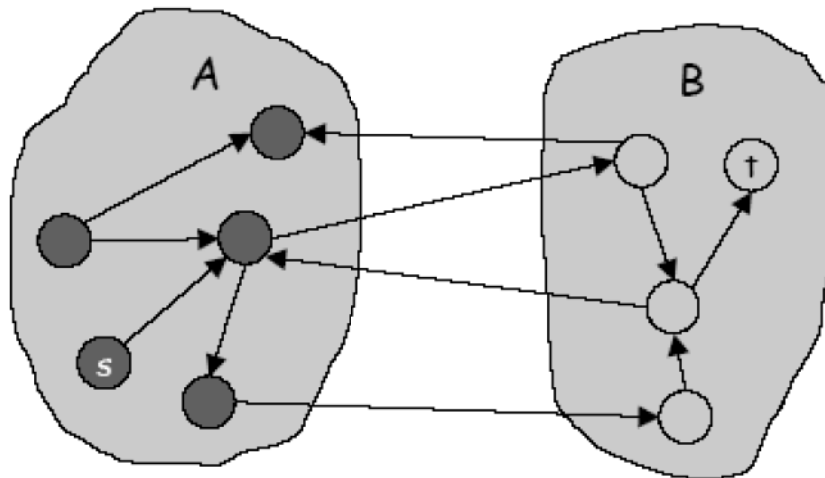


算法分析

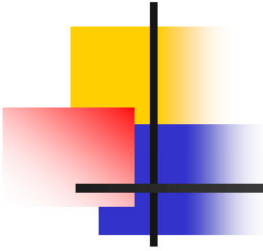
- 证明：(与最大流最小割定理证明思路一样)
在 Δ -缩放阶段, 存在割 (A, B) 使得 $\text{cap}(A, B) \leq v(f) + m \Delta$, 采用构造法。
 - 选择 A 是 $G_f(\Delta)$ 中从 s 出发可达的顶点集合
 - 根据定义 $s \in A; t \notin A, \in B$.
如果 $e=(u,v), u \in A, v \in B$, 那么 $c(e) < f(e) + \Delta$;
如果 $e'=(u',v'), u' \in B, v' \in A$, 那么 $f(e') < \Delta$.

算法分析

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &\geq \sum_{e \text{ out of } A} (c(e) - \Delta) - \sum_{e \text{ in to } A} \Delta \\ &= \sum_{e \text{ out of } A} c(e) - \sum_{e \text{ out of } A} \Delta - \sum_{e \text{ in to } A} \Delta \\ &\geq \text{cap}(A, B) - m\Delta . \end{aligned}$$

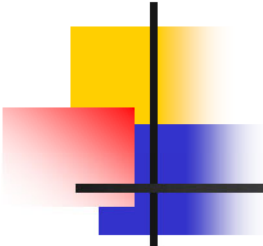


original network



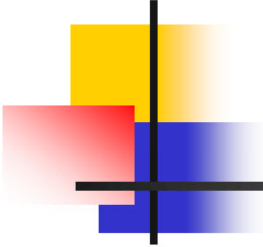
算法分析

- 命题7.19 在一个缩放阶段增广次数至多是 $2m$.
- 证明：考虑缩放阶段 Δ , 令 f_p 是前一缩放阶段结束时的流。那时采用 $\Delta' = 2\Delta$ 作为参数，于是最大流 f^* 至多是 $v(f_p) + 2m\Delta$, 因此至多可能存在 $2m$ 次增广。



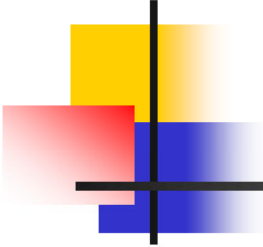
算法分析

- 一次增广用 $O(m)$ 时间(包括建立图, 找到合适路径)
- 缩放次数: 至多 $1 + \lceil \log_2 C \rceil$
- 缩放阶段增广次数: 至多 $2m$
- 定理7.20 在具有 m 条边和整数容量的图中, 缩放最大流算法找最大流至多用 $2m(1 + \lceil \log_2 C \rceil)$ 次增广, 于是可在 $O(m^2 \log C)$ 时间内运行。



算法分析

- 一般的**Ford-Fulkerson**算法需要与容量的数量级成正比的时间；
- 这里给出的缩放算法只需要与说明问题输入的容量所需字节数成正比的时间
- 缩放算法运行在**输入规模**(边数及容量的数字表示)的多项式时间

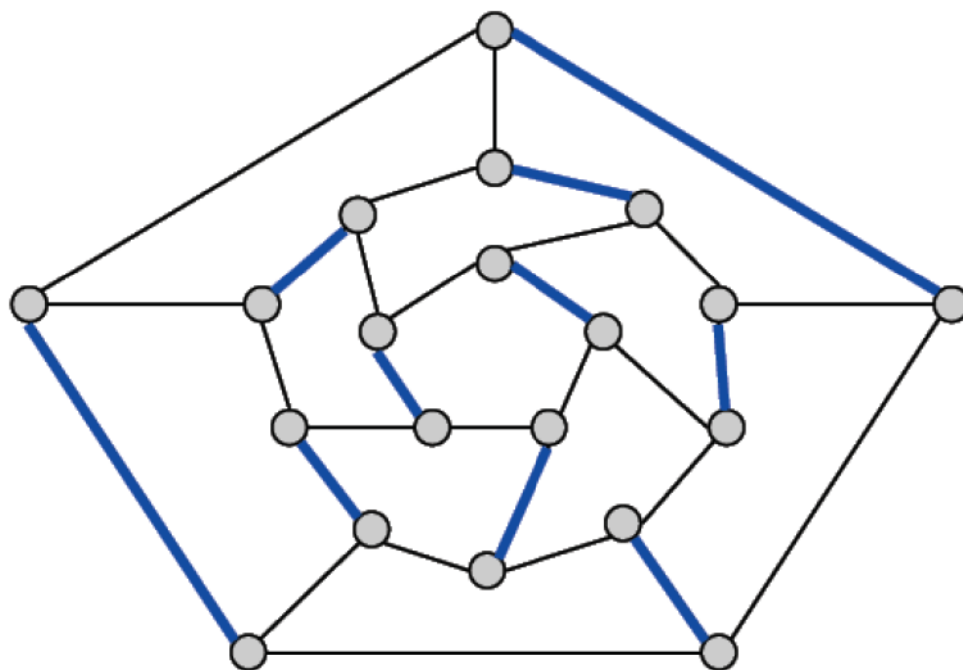


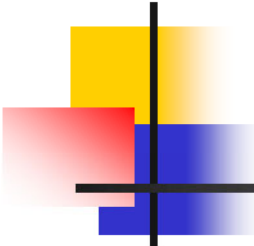
推广：强多项式算法

- [Edmonds-Karp 1972, Dinitz 1970]
- 存在强多项式算法
- 仅仅是边数 m , 顶点数 n 的多项式
- 每次迭代选择具有**最少边数**的增广路径
- $O(mn)$
- 其他的一些改进复杂度
 $O(mn \log n), O(n^3), \dots$

7.5 二分匹配问题

- 输入：无向图 $G = (V, E)$.
- $M \subseteq E$ 是一个匹配，如果每个结点至多出现在 M 中的一条边中。
- 最大匹配：寻找具有最大数目的匹配

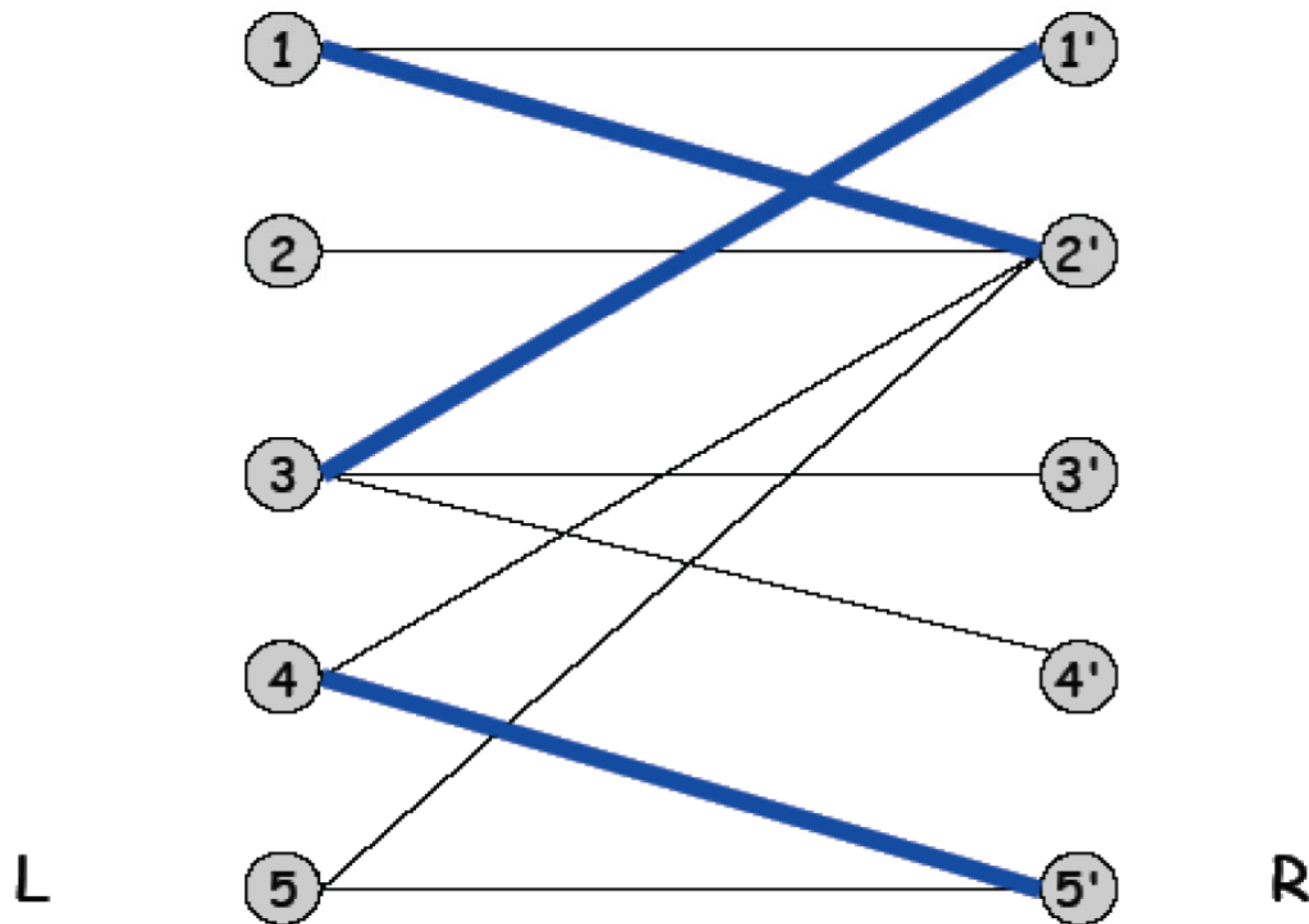




二分匹配问题

- **二部图** $G=(V,E)$ 是一个无向图，它的结点集合可以被划分成 $V = L \cup R$ ，并具有下述性质：每条边 $e \in E$ 有一个端点在 L 中，另一个端点在 R 中。
- 二分匹配。
 - 输入：无向，二部图, $G = (L \cup R, E)$.
 - $M \subseteq E$ 是一个**匹配**，如果每个结点至多出现在 M 中的一条边中。
 - 最大匹配：寻找具有最大数目的匹配

二分匹配问题

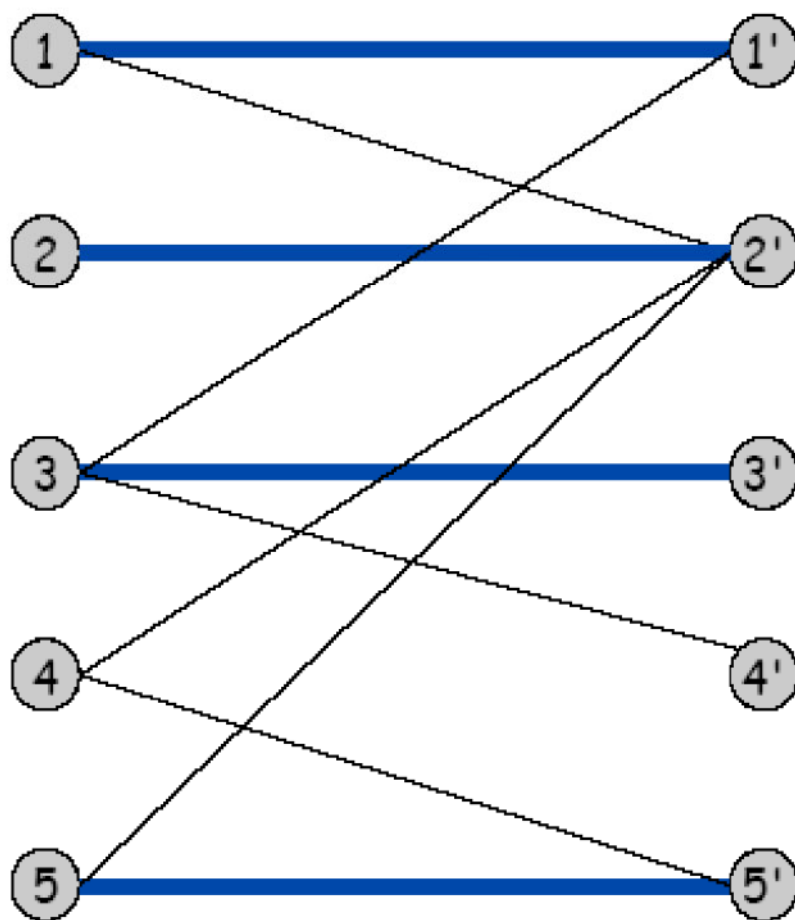


匹配

1-2', 3-1', 4-5'

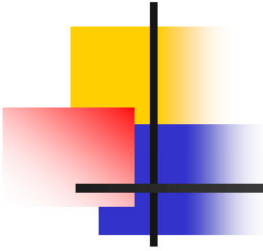
是最大匹配吗？

二分匹配问题



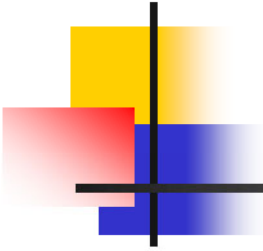
最大匹配

1-1', 2-2', 3-3' 4-4'



二分匹配问题

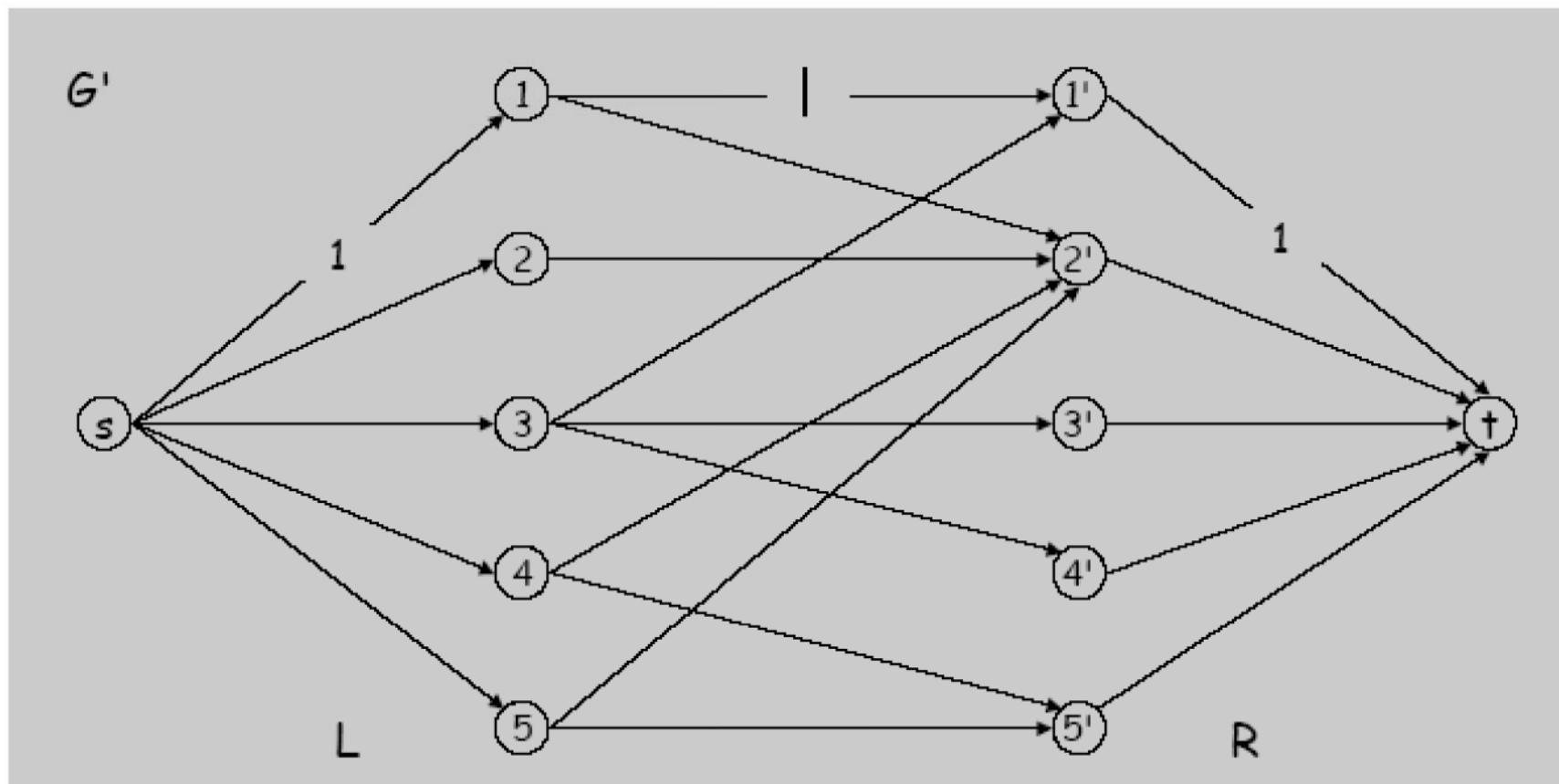
- 二分匹配问题看起来与流网络有一定类似的地方
- 这里将应用流网络的相关成型算法
- 首先构造一个流网络，满足需要的容量条件，守恒条件

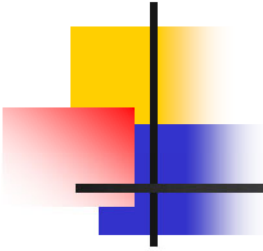


二分匹配问题

- 最大流的构造.
 - 构造图 $G' = (L \cup R \cup \{s, t\}, E')$.
 - 连接原图L到R的每条边, 每条边赋予单位容量.
 - 增加一个源点s, 从s到L中的每个结点连接一条边, 每条边赋予单位容量.
 - 增加一个终点 t, 从R中的每个结点到t连接一条边, 每条边赋予单位容量.

二分匹配问题

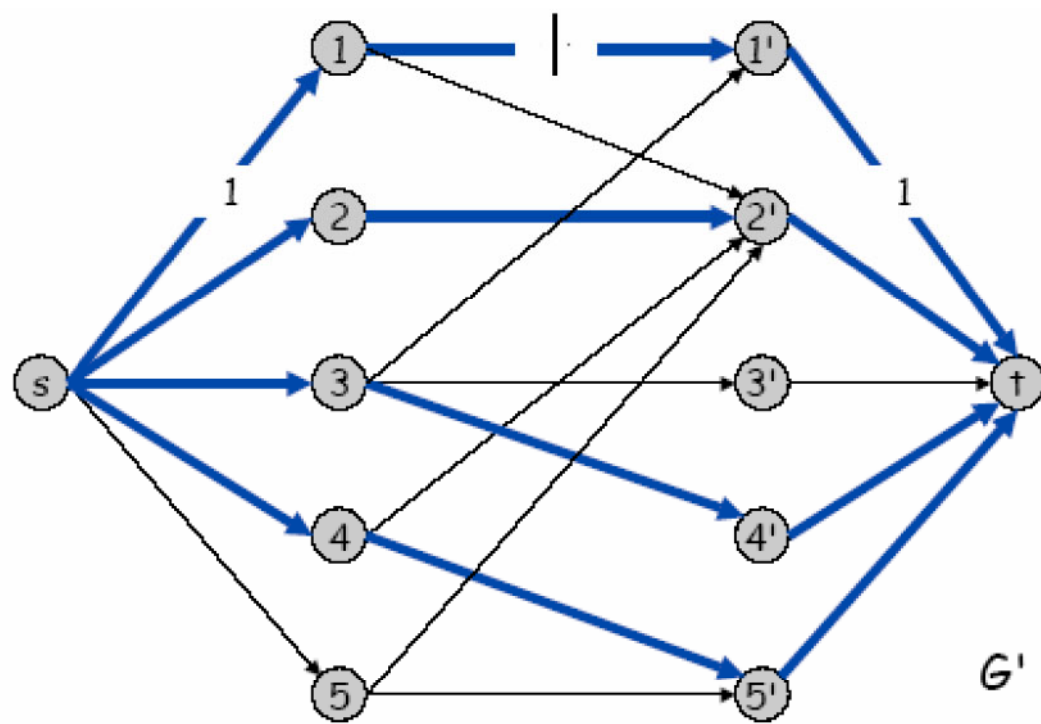
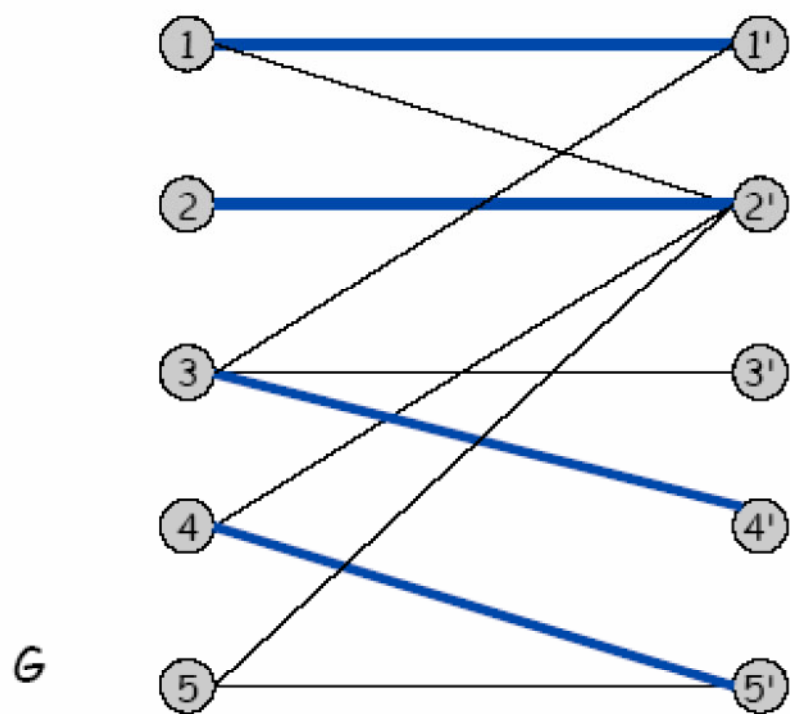


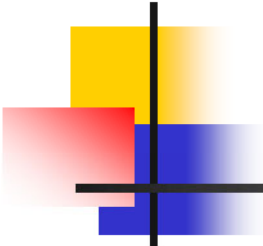


二分匹配问题

- 现在计算这个网络 G' 的最大 s - t 流，我们发现这个流的值等于 G 中最大匹配的大小。
- 定理. G 中最大匹配的数目与所定义的 G' 中最大流值相同.
- 证明:
 - 设 G 中最大匹配集合是 M ，其数目是 k .
 - 于是可以构造一个流 f , 每一条边从 s 出发，携带一个单位的容量.
 - f 是一个流，而且流值为 k .
 - 所以 G' 中最大流值 $\geq$ 最大匹配数目;

二分匹配问题





二分匹配问题

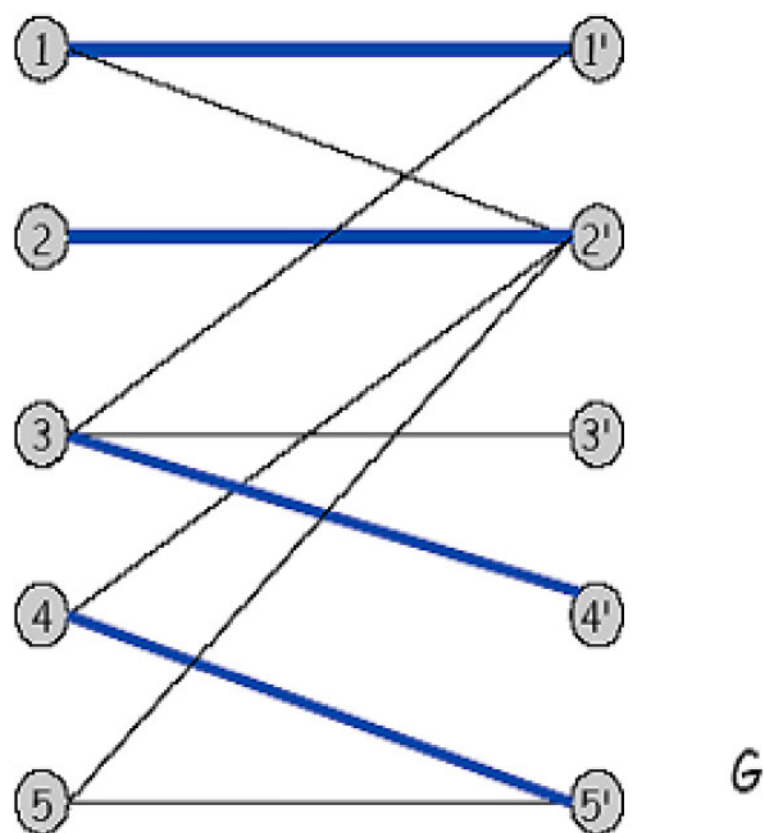
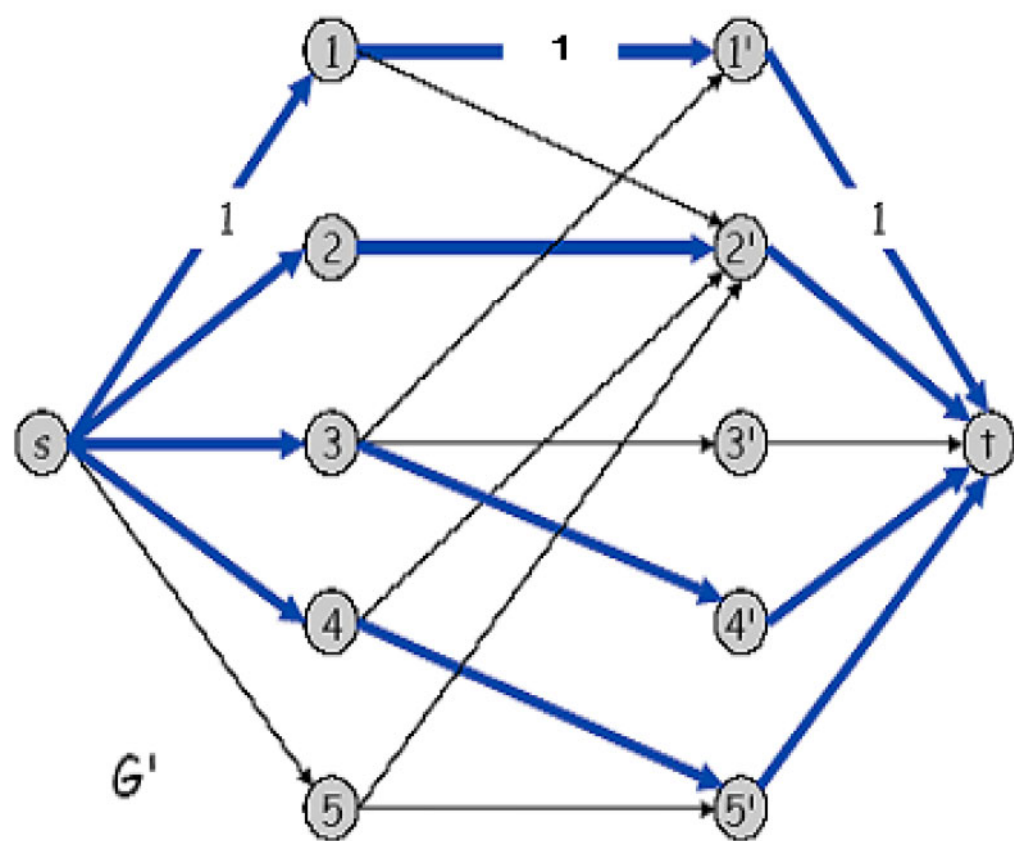
设 f 是 G' 最大流，其值为 k .

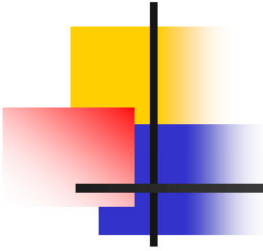
- 考虑集合 M : 从 L 到 R 权值为 1 的边的集合,
i.e., $f(e) = 1$.
 - 可以发现每个节点至多在 M 的一条边中
 - $|M| = k$: 割 $(L \cup s, R \cup t)$ 就是一个匹配

所以最大匹配的数目 $\geq$ 最大流值

因此定理成立。

二分匹配问题





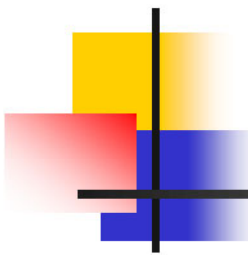
二分匹配问题： 界定运行时间

- 令 $n=|L|=|R|$, m 是 G 的边数

- 时间复杂度？

注意到 $C=|L|=n$, 根据以前 $O(mC)$ 的界

- 定理7.38 可以用Ford-Fulkerson算法在 $O(mn)$ 时间内找到二部图中的一个最大匹配。

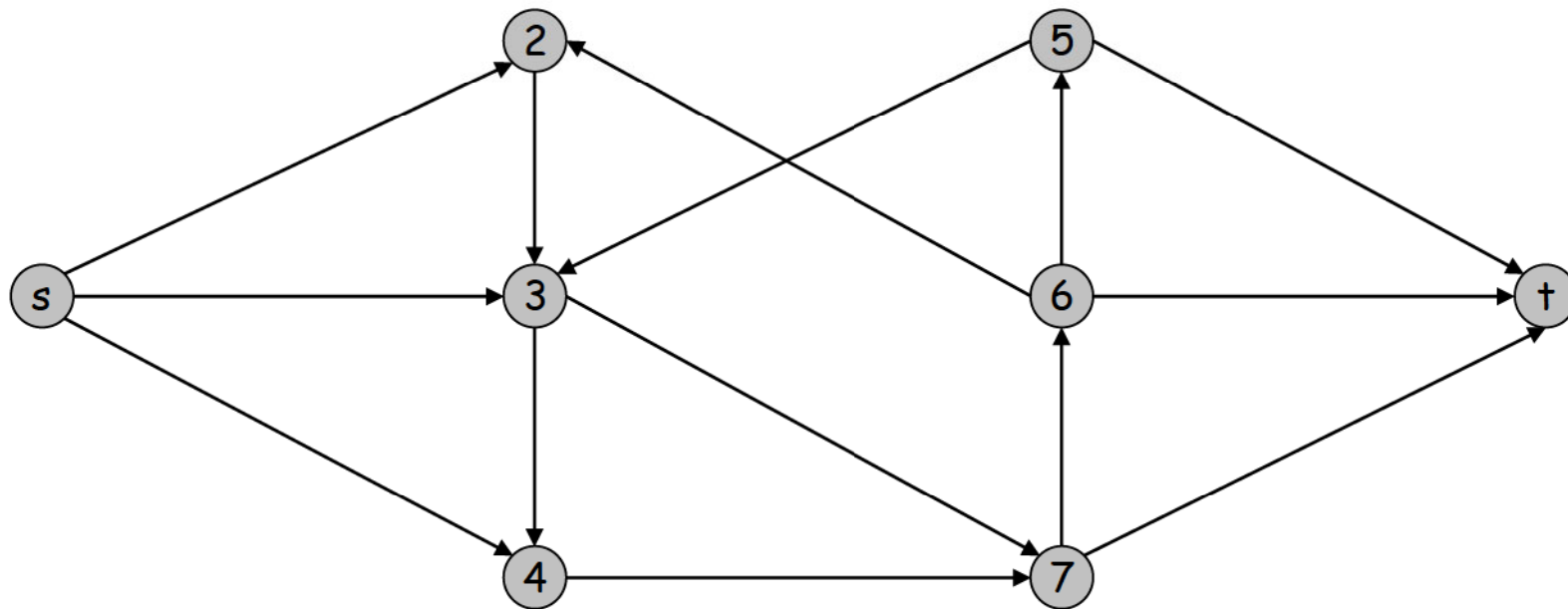


7.6 有向与无向图中的不交路径

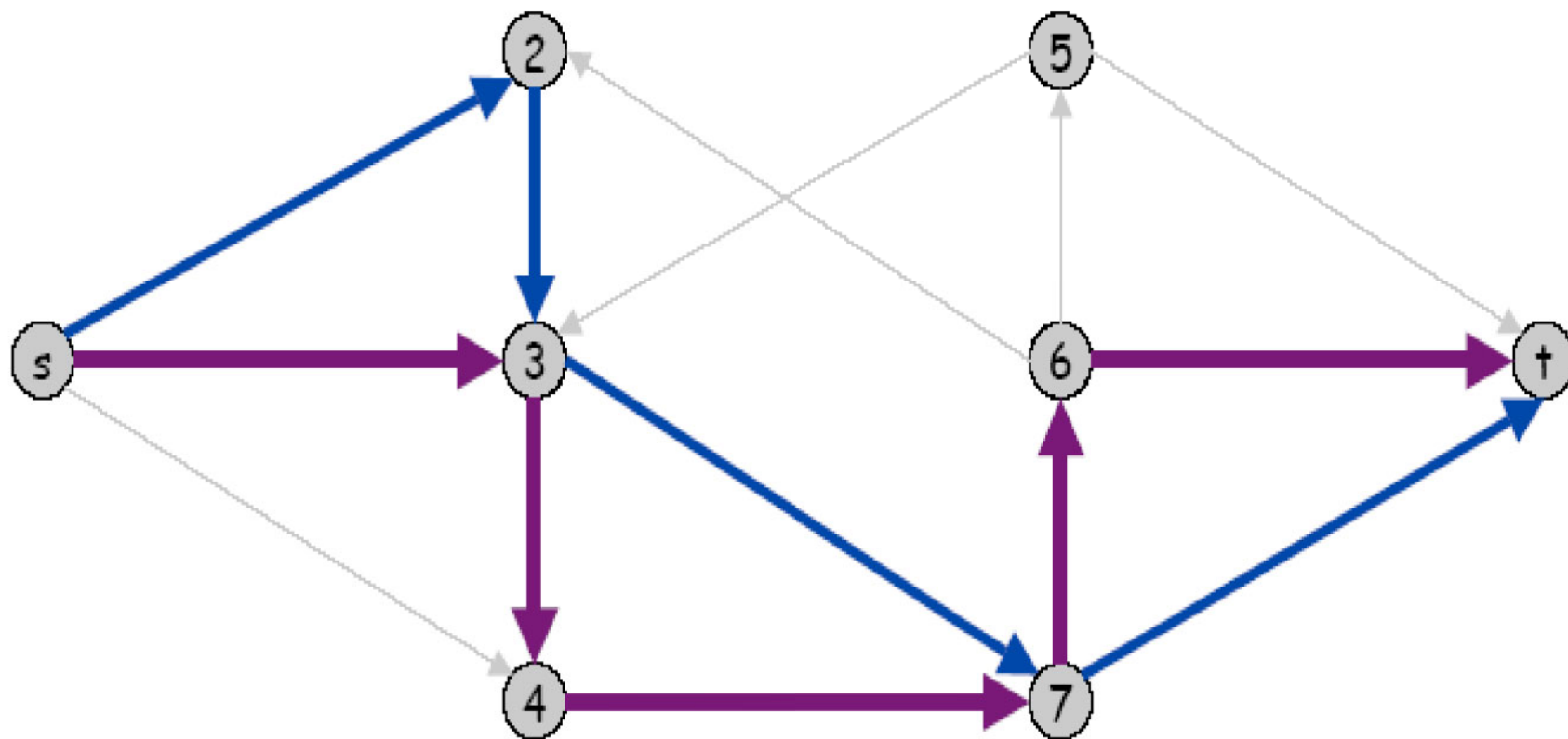
- 定义 两条路径称为是**不交路径**，如果它们没有公共边。
- 不交路径问题. 给定图 $G = (V, E)$ 以及两个结点 s, t , 寻找最大的不相交 s - t 路径数目.
- 例子: communication networks.

不交路径

寻找不交路径的最大数目



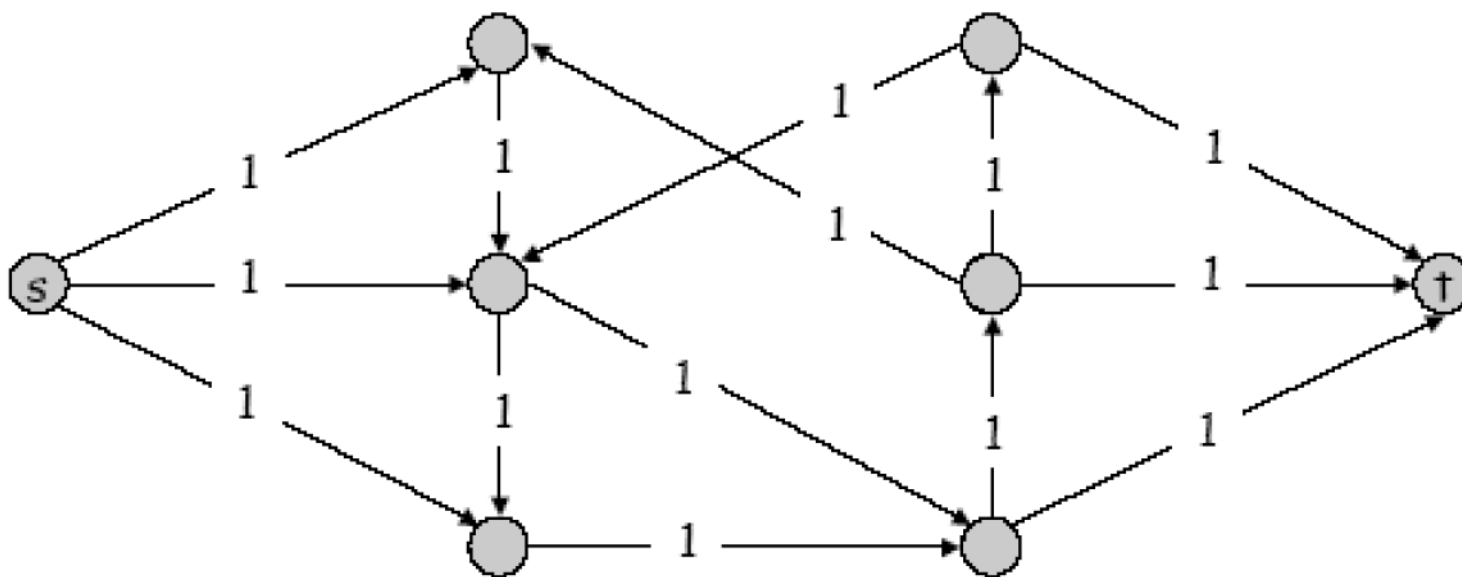
不交路径

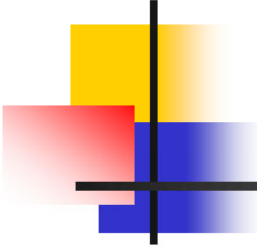


不交路径的最大数目：2

不交路径

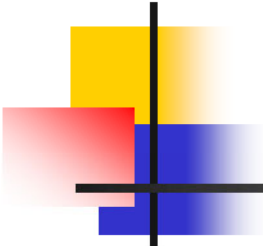
- 给定图 $G=(V,E)$ 以及不同的结点 s,t ,定义一个流网络: 其中 s,t 分别是源点, 汇点, 且每条边容量为1.





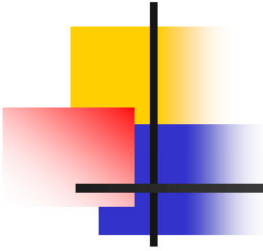
不交路径

- 假设存在 **k** 条边不交的 **s - t** 路径，我们可以使每一条这样的路径携带一个单位的流；
- 对任何这种路径的边 **e** 设定 **$f(e)=1$** ,其他的边 **$f(e')=0$** .
- 命题**7.41** 如果在有向图 **G** 中从 **s** 到 **t** 存在 **k** 条边不交的路径，那么 **G** 中的最大 **s - t** 流至少是 **k** .



不交路径

- 最大流 f 是整数值，而且所有的边容量以1为界，因此每条在 f 下携带流的边上恰好有一个单位的流
- 命题7.42 如果 f 是值为 v 的0-1流，那么具有流值 $f(e)=1$ 的边的集合包含一组 v 条(边不交)路径。



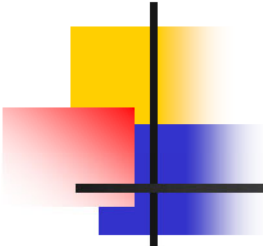
不交路径

- 证明：采用归纳法(f 携带流的边数)证明此命题
- $V=0$, 容易说明
- $V \geq 1$, 那么存在 (s, u) 携带一个单位的流，由守恒条件， $(u, v), \dots$, 生成延续的路径

case 1: 到达 t , 可以应用归纳假设;

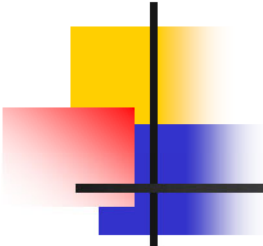
case 2: 在某个结点 v 处形成圈 C , 把圈上的流值减少到0, 新的流值不变, 边数更少

(Fig 7.12)



不交路径

- 定理7.43 在有向图 G 中从 s 到 t 存在 k 条边不交路径当且仅当在 G 中 s - t 流的最大值至少是 k .
- 运行时间
- $C \leq |V| = n$, 因此利用一般的Ford-Fulkerson算法可以在 $O(mn)$ 时间内得到一个整数最大流。



不交路径

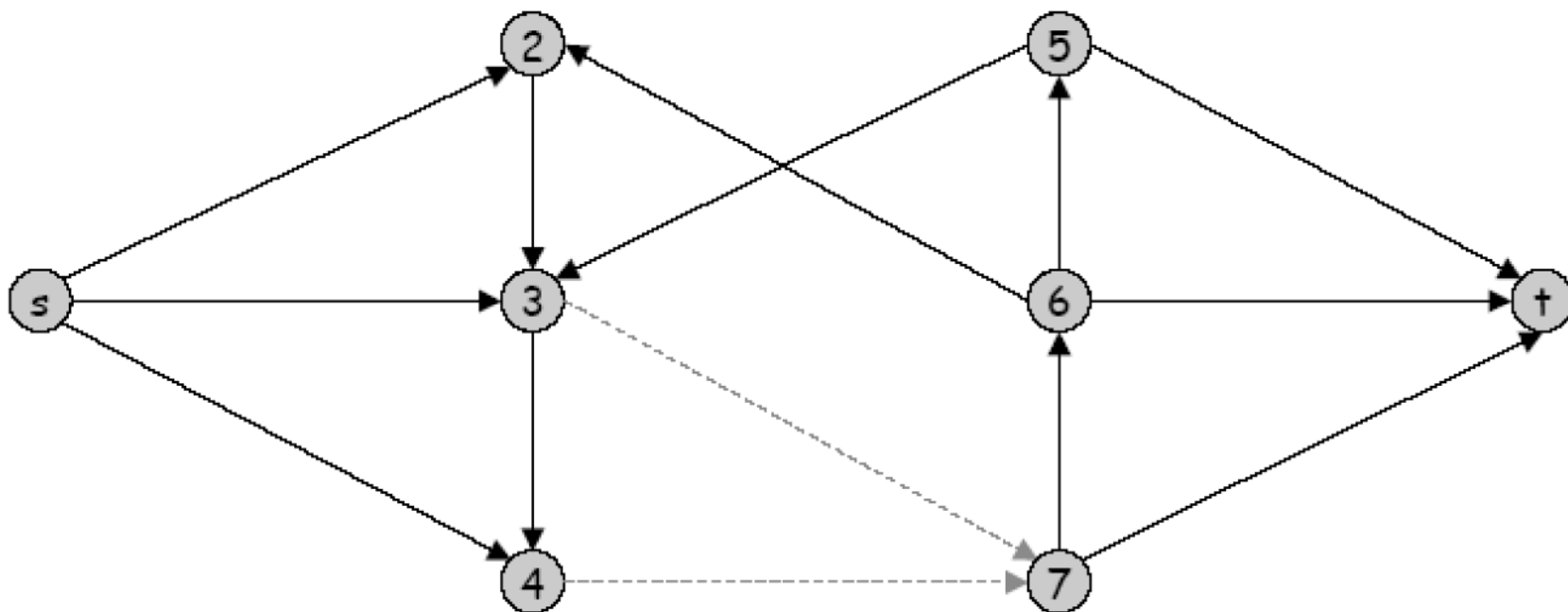
- 定理7.44 可以用Ford-Fulkerson算法在 $O(mn)$ 时间内在有向图 G 中找到一组最大的边不交的 s - t 路径。

相关问题

- 给定图 $G = (V, E)$ ，以及源点 s 汇点 t ，寻找最少数目的边分离 t 与 s 。

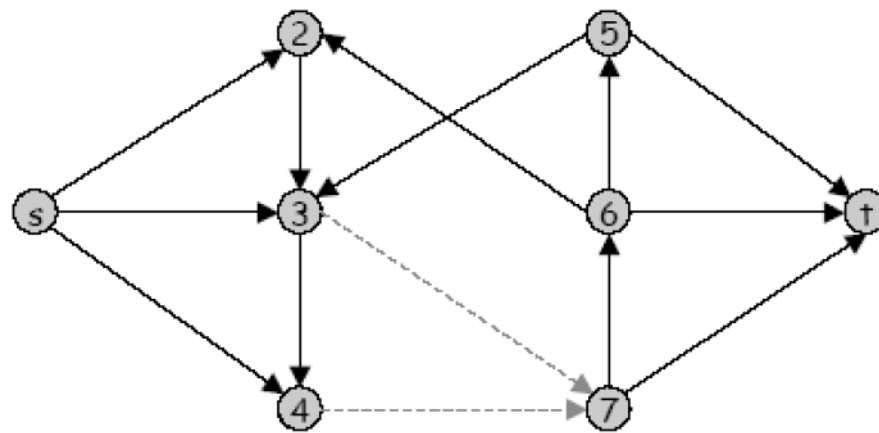
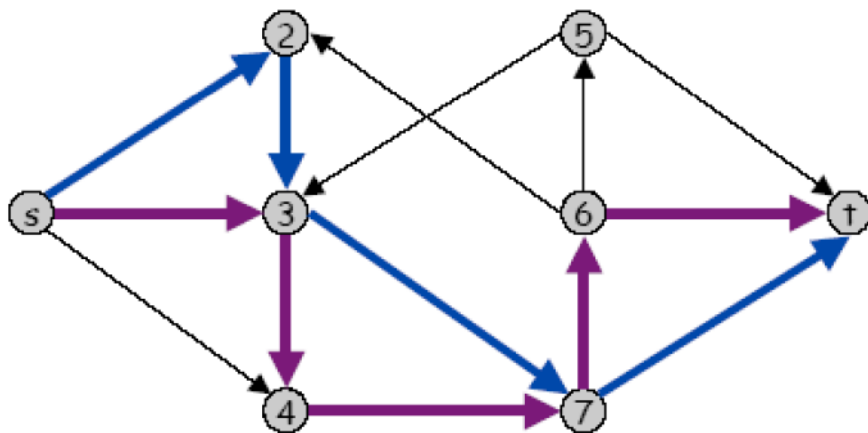
不交路径

- 如果从图 G 中拿走一组边 $F \subseteq E$ 以后，图中一条 s - t 路径也没剩下，我们就说 F 分离 s 与 t 。(所有 s - t 路径至少使用 F 中的一条边)



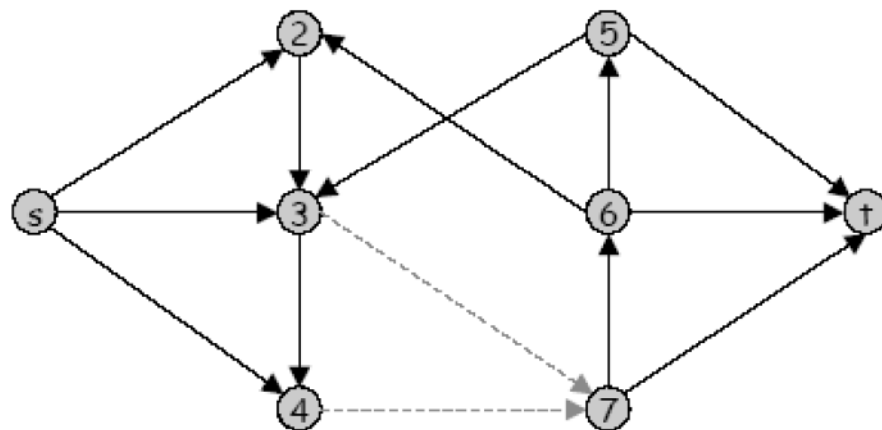
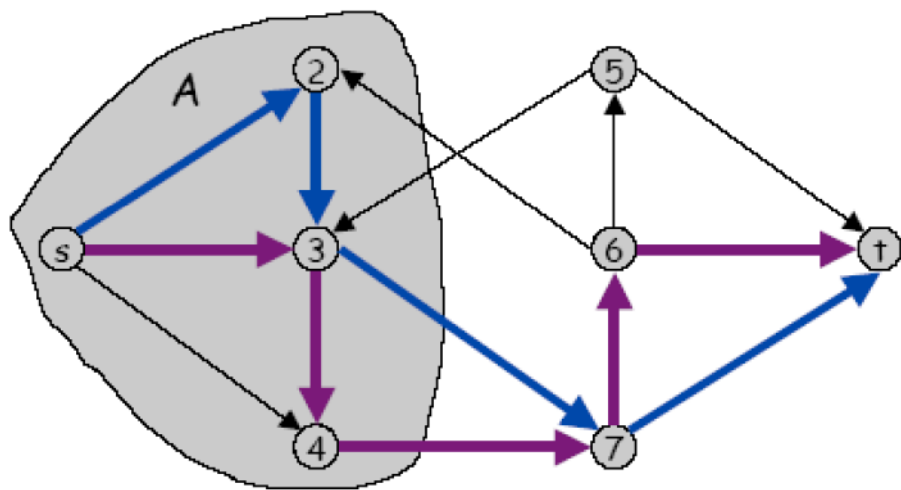
不交路径

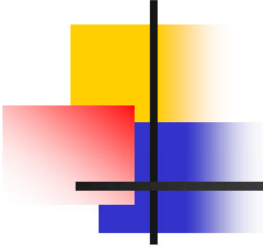
- 定理7.45 [Menger 1927] 在每个具有结点s与t的有向图中，边不交s-t路径的最大数目等于为分离s与t所需移走的最少边数。
- 证明：如果移走边集 $F \subseteq E$ 就分离了s与t,那么边不交的s-t路径数目至多是 $|F|$, 于是边不交s-t路径的最大数目 $\leq$ 分离s与t所需移走的最少边数;



不交路径

- 设边不交s-t路径的最大数目是 v , 那么最大s-t流值也是 v , 根据最大流最小割定理, 存在cut (A, B) 达到容量 v . 设F是从A到B的边集合, $|F| = v$, 那么F分离了t与s。所以分离s,t所需的最少边数 $\leq$ 边不交s-t路径的最大数目

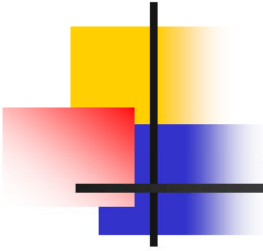




无向图的不交路径

考虑无向图的情形

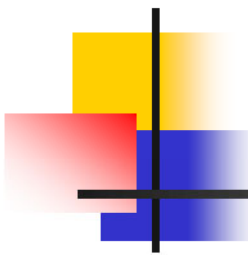
- 仍然可以借鉴有向图的想法
- 把 G 中的每条无向边 (u,v) 用两条有向边 $(u,v), (v,u)$ 代替
- 注意两条路径在有向图中可能是边不交的，但是无向图中却可能共享一条边



无向图的不交路径

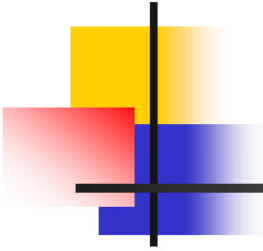
- 在任何网络中总存在一个最大流，它在每一对相反的有向边中**至多**使用一条。

命题7.46 在任何流网络中存在一个最大流 f , 其中对所有相反的有向边 $e=(u,v)$ 与 $e'=(v,u)$, 有 $f(e)=0$ 或 $f(e')=0$. 如果流网络的容量是整数, 那么也存在这样一个整数的最大流。



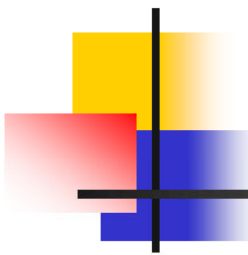
无向图的不交路径

- 证明： 考虑任何最大流 f ,我们修改以满足需要的条件。假定 $e=(u,v)$ 和 $e'=(v,u)$ 是相反的有向边，并且 $f(e) \neq 0, f(e') \neq 0$
设 $\delta = \min(f(e), f(e'))$ ，通过在 e, e' 上把流值减小 δ ，所得到的流 f' 与 f 有相同的值，并满足所需条件。



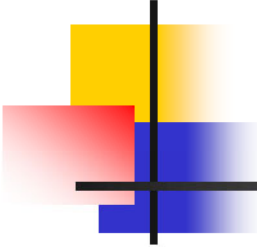
无向图的不交路径

- 类似的我们可以采用有向图中的**Ford-Fulkerson**算法，以及路径分解过程来得到无向图中的边不交路径。
- **定理7.47** 在无向图 G 中存在 k 条从 s 到 t 的边不交的路径，当且仅当在 G 对应的有向图 G' 中 s - t 流的最大值至少是 k . 此外，可以用**Ford-Fulkerson**算法在 $O(mn)$ 时间内找到无向图 G 中的一组最大的边不交的 s - t 路径。



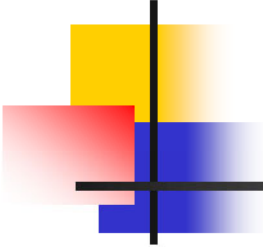
无向图的不交路径

- 定理7.48 在每个具有节点 s 与 t 的无向图中，边不交的 s - t 路径的最大数目等于为分离 s 与 t 所需要移走的最少边数。



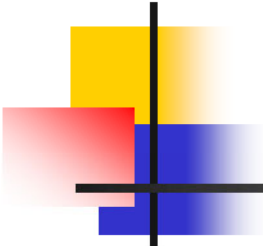
7.7 对最大流问题的推广

- 问题描述：带需求的流通
- 在实际网络中，可能会有多个源点，汇点；
- 比如网络代表公路或者铁路系统，其中我们想把产品从有供给的工厂运送到有应用需求的临售商店。我们关注是否存在有效的供给来满足所有的需求。



最大流问题的推广

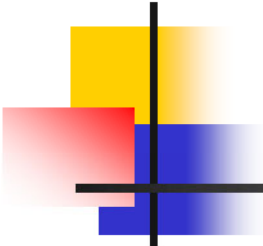
- 带需求的流通.
 - 有向图 $G = (V, E)$.
 - 边容量 $c(e)$, $e \in E$.
 - 结点供应值, 与需求值 $d(v)$, $v \in V$.
 - $d(v) > 0$, 表示需求结点; $d(v) < 0$ 表示供给结点



最大流问题的推广

- 我们说带需求 $\{d(v)\}$ 的流通是一个函数 f , 它对每条边分配一个非负的实数, 并且满足下面两个条件:
 - i. (容量条件) 对于每个 $e \in E: 0 \leq f(e) \leq c(e)$
 - ii. (需求条件) 对于每个 $v \in V$, $\sum_{e \text{ in to } v} f(e) - \sum_{e \text{ out of } v} f(e) = d(v)$

流通问题: 给定 (V, E, c, d) , 是否存在一个满足条件i, ii, 的流通?

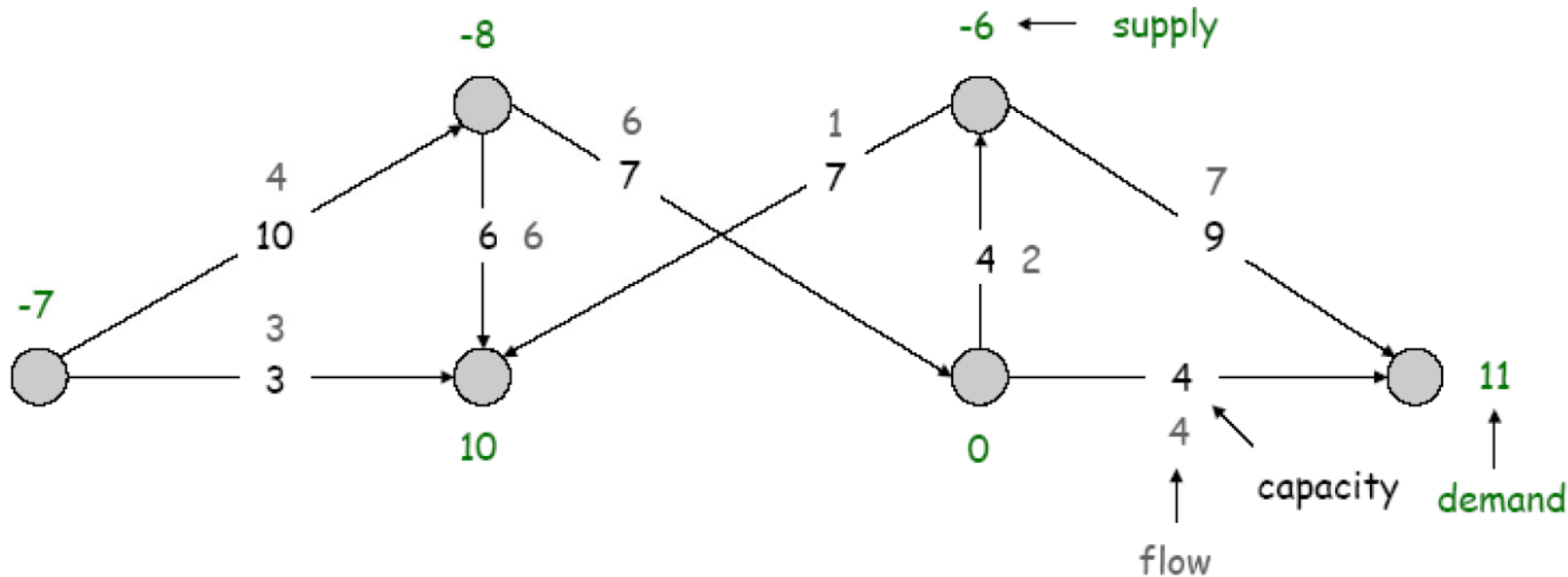


最大流问题的推广

- 直觉经验告诉我们，为了存在一个可行流通，必须满足总供给等于总需求。
- 命题7.49 如果存在一个带需求 $\{d(v)\}$ 的可行流通，那么 $\sum d_v = 0$ 。
- 证明： $\sum_v d_v = \sum_v (f^{in}(v) - f^{out}(v))$ 。对于每条边 $e=(u,v)$ ， $f(e)$ 恰好计算两次，相互抵消。

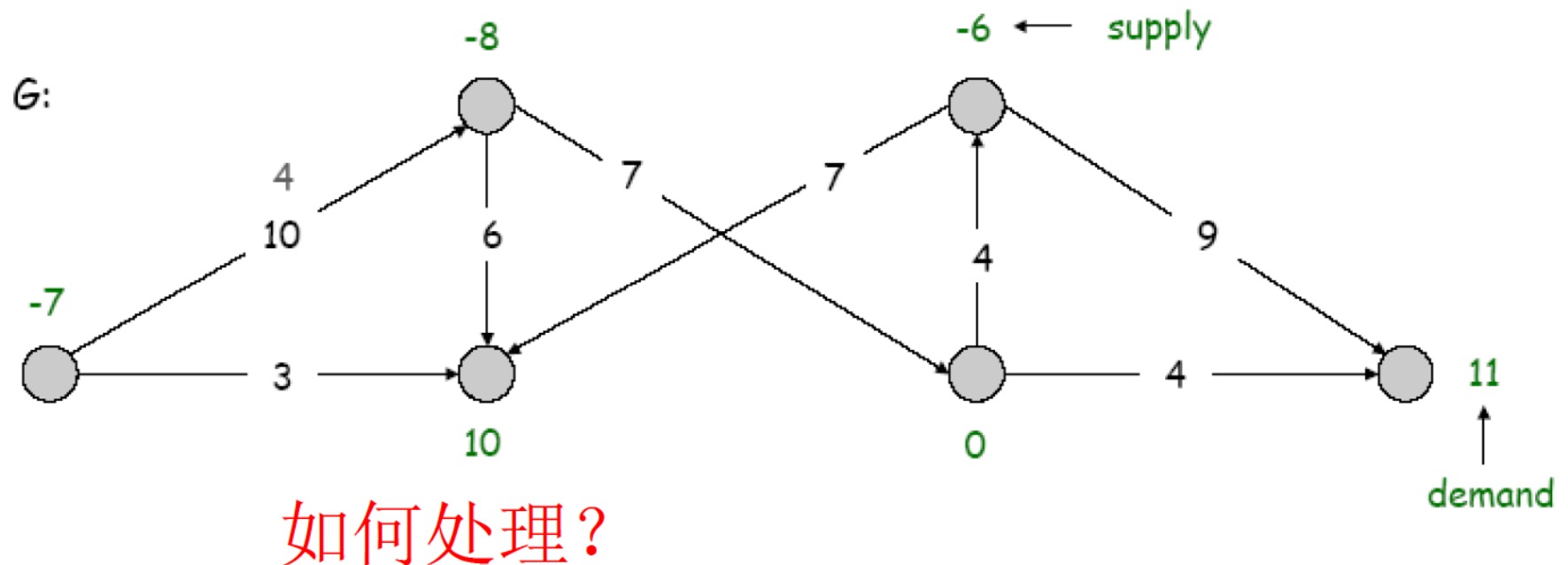
最大流问题的推广

$$\sum_{v: d(v) > 0} d(v) = \sum_{v: d(v) < 0} -d(v) =: D$$



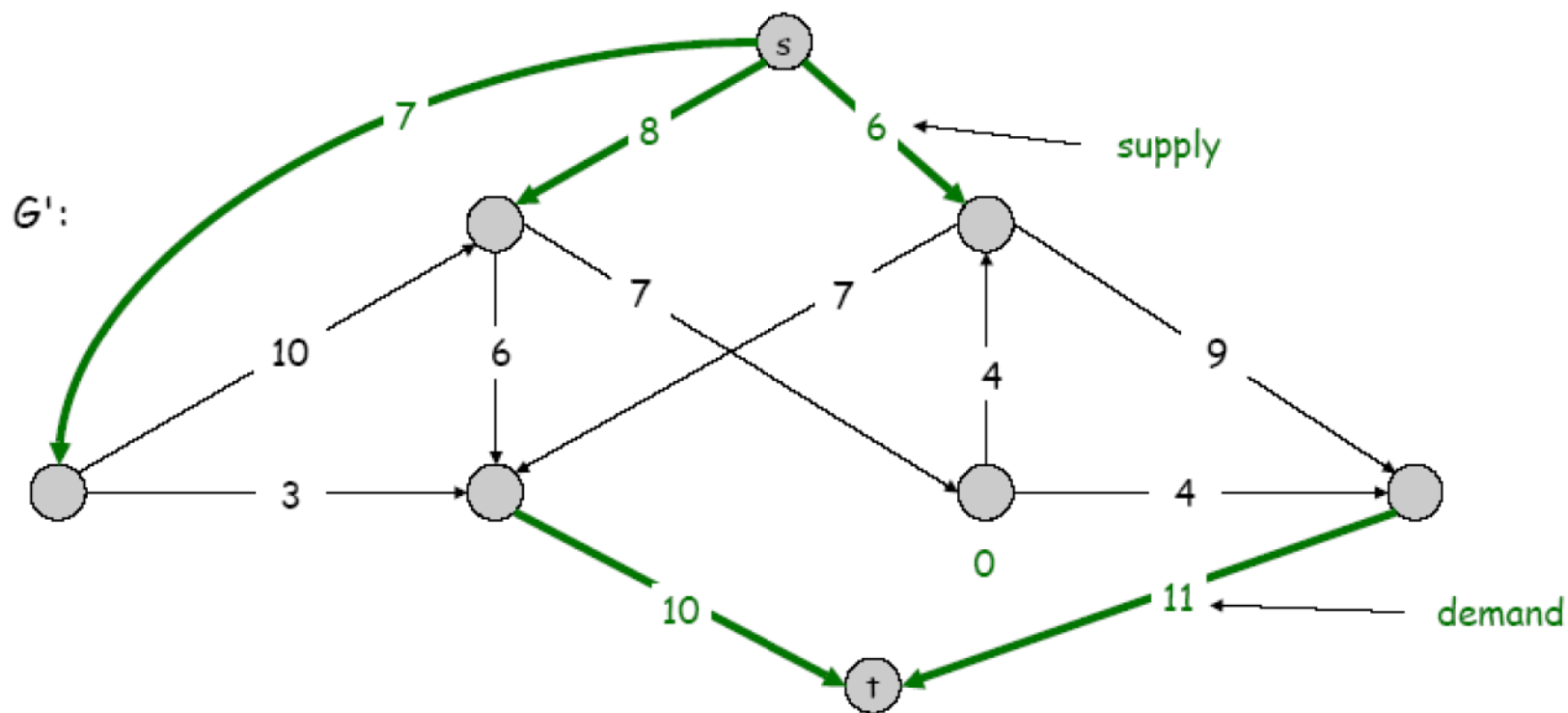
最大流问题的推广

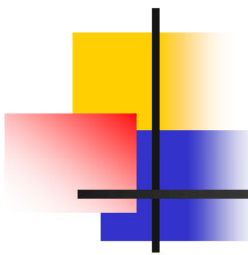
- 把问题转换成最大s-t流问题
- 需要把多个源点转换成一个源点，多个汇点转换成一个汇点



最大流问题的推广

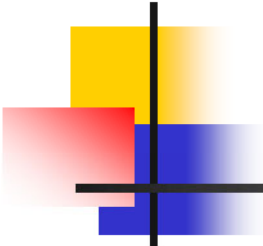
- 增加新的源点 s ，以及汇点 t .
- 对于每个 $d(v) < 0$ 的结点 v , 增加边 (s, v) , 容量为 $-d(v)$.
- 对于每个 $d(v) > 0$ 的结点 v , 增加边 (v, t) , 容量为 $d(v)$.
- G 存在一个流通当且仅当 G' 中最大流值为 D .





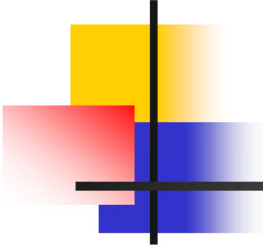
最大流问题的推广

- 定理7.50 G 中存在一个带 $\{d_v\}$ 的可行流通，当且仅当 G' 的最大 s^*-t^* 流值为 D . 如果 G 中所有的容量和需求都是整数，且存在一个可行流通，那么存在一个整数值的可行流通。



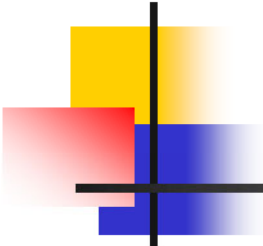
最大流问题的推广

- 不存在可行流通图的特征？
- 特征： 给定 (V, E, c, d) , 不存在一个流通，当且仅当如果存在一个结点划分 (A, B) 满足 $\sum_{v \in B} d_v > c(A, B)$
- 命题7.51 图 G 有一个带需求 $\{d_v\}$ 的可行流通当且仅当对所有的割 (A, B) , $\sum_{v \in B} d_v \leq c(A, B)$.



最大流问题的推广

- 许多应用中，我们还想使得某个流使用某些确定的边
- 在边上设置下界来实现
- 可行流通.
 - 有向图 $G = (V, E)$.
 - 每条边容量限制 $c(e)$ 以及下界限制 $\ell(e)$, $e \in E$.
 - 结点供应需求 $d(v)$, $v \in V$.



最大流问题的推广

定义： 一个流通是满足下面两个条件的函数：

i. (容量条件) 对于每个 $e \in E$:

$$\ell(e) \leq f(e) \leq c(e)$$

ii. (需求条件) 对于每个 $v \in V$,

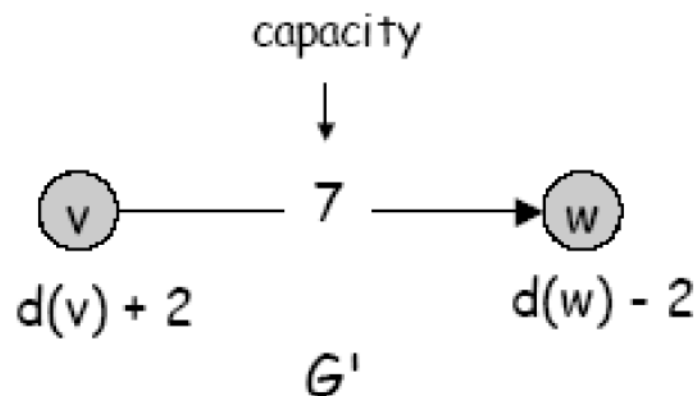
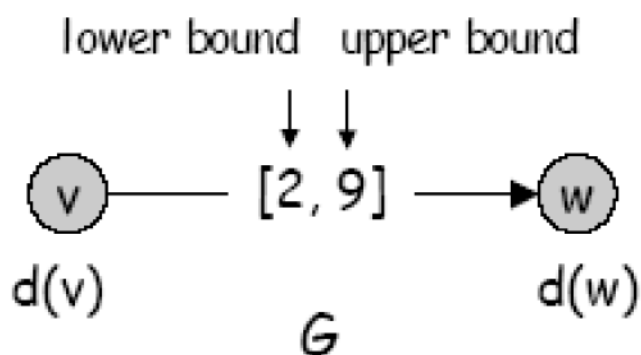
$$\sum_{e \text{ in to } v} f(e) - \sum_{e \text{ out of } v} f(e) = d(v)$$

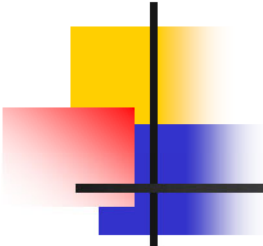
具有下界的流通问题：

给定 (V, E, ℓ, c, d) , 是否存在一个流通?

最大流问题的推广

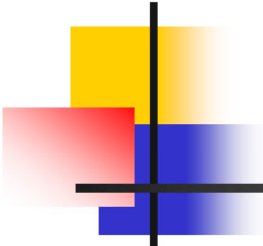
- 思路：转换成我们熟悉的问题：没有下界
 - 沿着边 e 发送 $\ell(e)$ 单位的流。
 - 更新结点的需求 d_v 。





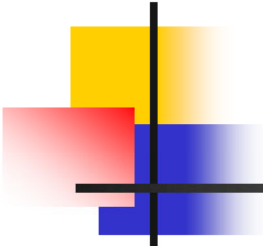
最大流问题的推广

- $G \rightarrow G'$
- 定理7.52 在 G 中存在可行流通当且仅当在 G' 中存在一个可行流通。如果 G 中所有的需求，容量，以及下界都是整数，并且存在可行流通，那么存在整数值的可行流通。
- 证明思路： $f(e)$ 是 G 中的一个流通，当且仅当 $f'(e) = f(e) - \ell(e)$ 是 G' 中的一个流通。



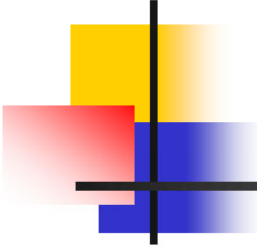
7.8 调查设计

- 数据挖掘领域中的一个主要问题是**客户偏爱模式**的研究
- 公司想要实施一项调查，向特定的**n**个顾客发放顾客调查表，试图确定人们喜欢那些产品



调查设计

- 调查方针：
 - 每个顾客将接收一组确定产品的问题
 - 一个顾客可能问到他曾经买过的产品
 - 对顾客 i 问到的产品数目在 c_i 与 c_i' 之间
 - 对每种产品 j , 问到这个产品的不同顾客数在 p_j 与 p_j' 之间
- 问题：是否存在一份调查表满足上面所有条件？

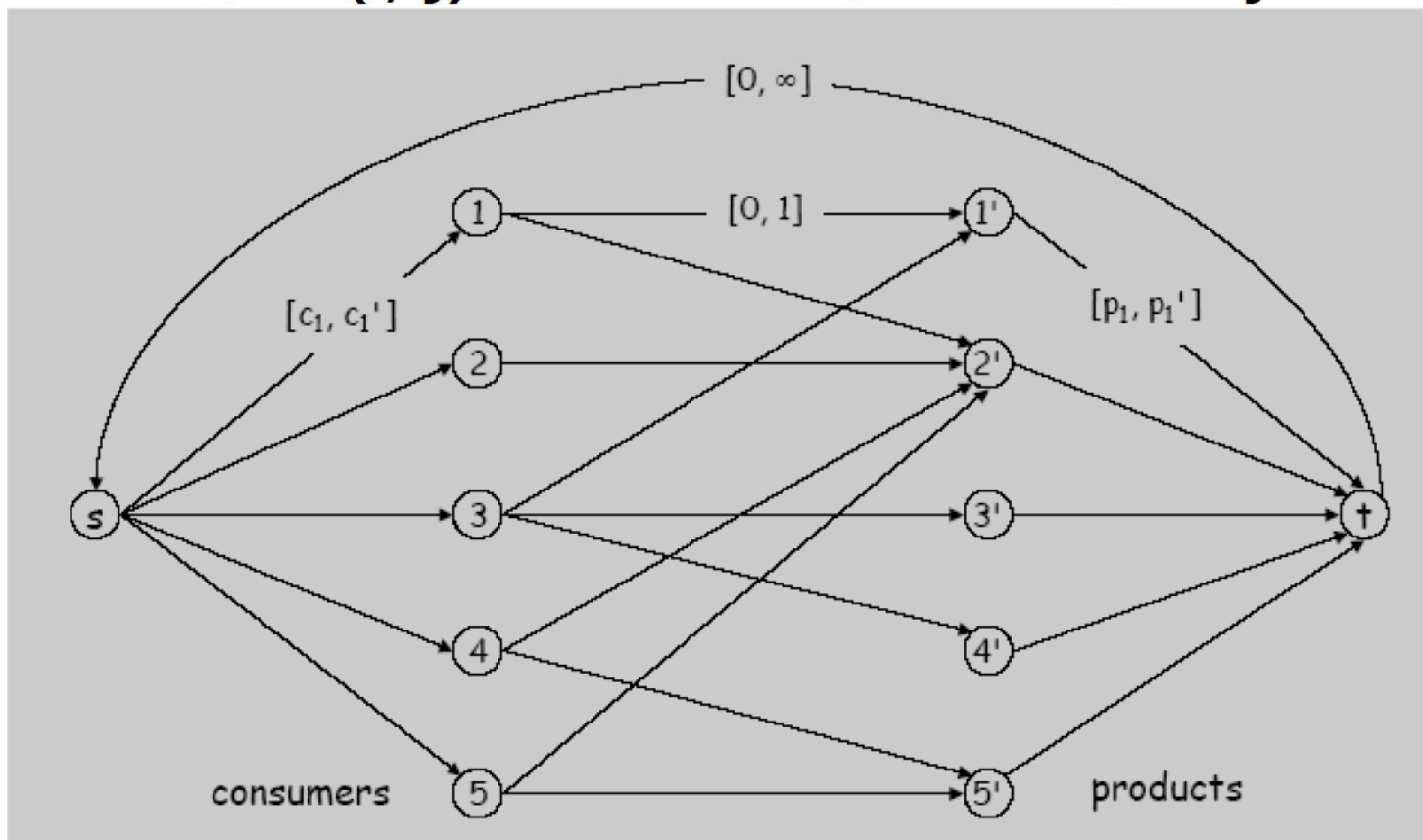


调查设计

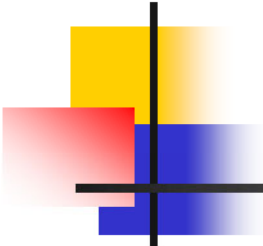
- 特殊情形: $c_i = c'_i = p_i = p'_i = 1$, 也就是二部图的完美匹配问题
- 设计算法: 转换成熟悉的问题
归约成在流网络 G' 上带需求和下界的流通问题

调查设计

- 包含边 (i, j) , 如果客户 i 被问到了产品 j .



$$t \rightarrow s : \sum_i c_i, \sum_i c_i'$$



调查设计

- 定理7.53 前面构造的图 G' 有一个可行流
通当且仅当存在一个可行方式来设计这个调查。
- 证明：如果客户 i 被问到了产品 j ,那么边 (i,j) 将携带一个单位的流。边 (s,i) 上的流是客户 i 被问到的问题数, (j,t) 上的流是产品 j 被问到的客户数; (t,s) 上的流是被问到的问题总数, 满足结点流守恒。